

Semana 13 - Reconhecimento Facial e Transferência de Estilo

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

- 1 O que é Reconhecimento Facial
- 2 One-shot Learning
- 3 Redes Siamesas
- 4 Verificação Facial e Classificação Binária
- 5 Neural Style Transfer
- 6 Função de Custo
- 7 Convoluções Generalizadas para 1D e 3D

O que é Reconhecimento Facial

- Sistemas de reconhecimento facial têm se tornado muito comuns, por exemplo, em fechaduras inteligentes
- Devem ser capazes também de identificar se a imagem da face corresponde a um ser humano “vivo” e não uma foto
- Duas tarefas frequentes: verificação e reconhecimento
- Na verificação facial (problema 1:1) temos uma imagem de entrada e o nome/ID de uma pessoa e a saída deve dizer se a imagem de entrada é daquela pessoa
- Já no reconhecimento (1:K, de K pessoas), temos uma base de K pessoas e uma imagem de entrada e a saída é o ID de alguma pessoa da base que foi reconhecida como sendo a imagem de entrada ou então “não reconhecido”

O que é Reconhecimento Facial

- Sistemas de reconhecimento facial têm se tornado muito comuns, por exemplo, em fechaduras inteligentes
- Devem ser capazes também de identificar se a imagem da face corresponde a um ser humano “vivo” e não uma foto
- Duas tarefas frequentes: verificação e reconhecimento
- Na verificação facial (problema 1:1) temos uma imagem de entrada e o nome/ID de uma pessoa e a saída deve dizer se a imagem de entrada é daquela pessoa
- Já no reconhecimento (1:K, de K pessoas), temos uma base de K pessoas e uma imagem de entrada e a saída é o ID de alguma pessoa da base que foi reconhecida como sendo a imagem de entrada ou então “não reconhecido”

O que é Reconhecimento Facial

- Sistemas de reconhecimento facial têm se tornado muito comuns, por exemplo, em fechaduras inteligentes
- Devem ser capazes também de identificar se a imagem da face corresponde a um ser humano “vivo” e não uma foto
- Duas tarefas frequentes: verificação e reconhecimento
- Na verificação facial (problema 1:1) temos uma imagem de entrada e o nome/ID de uma pessoa e a saída deve dizer se a imagem de entrada é daquela pessoa
- Já no reconhecimento (1:K, de K pessoas), temos uma base de K pessoas e uma imagem de entrada e a saída é o ID de alguma pessoa da base que foi reconhecida como sendo a imagem de entrada ou então “não reconhecido”

O que é Reconhecimento Facial

- Sistemas de reconhecimento facial têm se tornado muito comuns, por exemplo, em fechaduras inteligentes
- Devem ser capazes também de identificar se a imagem da face corresponde a um ser humano “vivo” e não uma foto
- Duas tarefas frequentes: verificação e reconhecimento
- Na verificação facial (problema 1:1) temos uma imagem de entrada e o nome/ID de uma pessoa e a saída deve dizer se a imagem de entrada é daquela pessoa
- Já no reconhecimento (1:K, de K pessoas), temos uma base de K pessoas e uma imagem de entrada e a saída é o ID de alguma pessoa da base que foi reconhecida como sendo a imagem de entrada ou então “não reconhecido”

O que é Reconhecimento Facial

- Sistemas de reconhecimento facial têm se tornado muito comuns, por exemplo, em fechaduras inteligentes
- Devem ser capazes também de identificar se a imagem da face corresponde a um ser humano “vivo” e não uma foto
- Duas tarefas frequentes: verificação e reconhecimento
- Na verificação facial (problema 1:1) temos uma imagem de entrada e o nome/ID de uma pessoa e a saída deve dizer se a imagem de entrada é daquela pessoa
- Já no reconhecimento (1:K, de K pessoas), temos uma base de K pessoas e uma imagem de entrada e a saída é o ID de alguma pessoa da base que foi reconhecida como sendo a imagem de entrada ou então “não reconhecido”

O que é Reconhecimento Facial

- O algoritmo de verificação é usado em cada pessoa no reconhecimento
- Mas o reconhecimento é bem mais difícil do que a verificação
- Se um sistema de verificação tem acurácia de 99%, mas no reconhecimento temos 100 pessoas, teríamos então 1% de erro para UMA pessoa, mas em 100 esse erro vai se propagando
- Precisamos então de uma acurácia ainda maior, como 99.9% ou até mais

O que é Reconhecimento Facial

- O algoritmo de verificação é usado em cada pessoa no reconhecimento
- Mas o reconhecimento é bem mais difícil do que a verificação
- Se um sistema de verificação tem acurácia de 99%, mas no reconhecimento temos 100 pessoas, teríamos então 1% de erro para UMA pessoa, mas em 100 esse erro vai se propagando
- Precisamos então de uma acurácia ainda maior, como 99.9% ou até mais

O que é Reconhecimento Facial

- O algoritmo de verificação é usado em cada pessoa no reconhecimento
- Mas o reconhecimento é bem mais difícil do que a verificação
- Se um sistema de verificação tem acurácia de 99%, mas no reconhecimento temos 100 pessoas, teríamos então 1% de erro para UMA pessoa, mas em 100 esse erro vai se propagando
- Precisamos então de uma acurácia ainda maior, como 99.9% ou até mais

O que é Reconhecimento Facial

- O algoritmo de verificação é usado em cada pessoa no reconhecimento
- Mas o reconhecimento é bem mais difícil do que a verificação
- Se um sistema de verificação tem acurácia de 99%, mas no reconhecimento temos 100 pessoas, teríamos então 1% de erro para UMA pessoa, mas em 100 esse erro vai se propagando
- Precisamos então de uma acurácia ainda maior, como 99.9% ou até mais

Outline

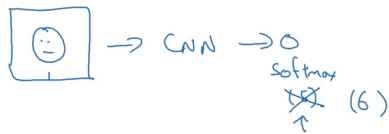
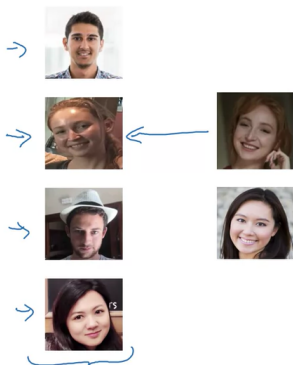
- 1 O que é Reconhecimento Facial
- 2 One-shot Learning**
- 3 Redes Siamesas
- 4 Verificação Facial e Classificação Binária
- 5 Neural Style Transfer
- 6 Função de Custo
- 7 Convoluções Generalizadas para 1D e 3D

One-shot Learning

- Um desafio importante na maioria dos sistemas de reconhecimento facial é que o modelo deve reconhecer uma pessoa dada uma única imagem do rosto dela (*one-shot learning*)
- Historicamente, algoritmos de *deep learning* costumam não funcionar bem nesse cenário

One-shot Learning

- Um desafio importante na maioria dos sistemas de reconhecimento facial é que o modelo deve reconhecer uma pessoa dada uma única imagem do rosto dela (*one-shot learning*)
- Historicamente, algoritmos de *deep learning* costumam não funcionar bem nesse cenário



One-shot Learning

- O algoritmo deve reconhecer uma pessoa, que tem uma única foto na base, assim como identificar quando uma pessoa não faz parte daquela base (intruso)
- Criar uma CNN com uma *softmax* de 5 classes na saída (4 pessoas + nenhum dos anteriores)?
- Não funciona bem devido ao tamanho pequeno do treinamento
- Além disso, se um novo funcionário chega, precisa retreinar toda a base (para uma *softmax* de 6 classes)

One-shot Learning

- O algoritmo deve reconhecer uma pessoa, que tem uma única foto na base, assim como identificar quando uma pessoa não faz parte daquela base (intruso)
- Criar uma CNN com uma *softmax* de 5 classes na saída (4 pessoas + nenhum dos anteriores)?
- Não funciona bem devido ao tamanho pequeno do treinamento
- Além disso, se um novo funcionário chega, precisa retreinar toda a base (para uma *softmax* de 6 classes)

One-shot Learning

- O algoritmo deve reconhecer uma pessoa, que tem uma única foto na base, assim como identificar quando uma pessoa não faz parte daquela base (intruso)
- Criar uma CNN com uma *softmax* de 5 classes na saída (4 pessoas + nenhum dos anteriores)?
- Não funciona bem devido ao tamanho pequeno do treinamento
- Além disso, se um novo funcionário chega, precisa retreinar toda a base (para uma *softmax* de 6 classes)

One-shot Learning

- O algoritmo deve reconhecer uma pessoa, que tem uma única foto na base, assim como identificar quando uma pessoa não faz parte daquela base (intruso)
- Criar uma CNN com uma *softmax* de 5 classes na saída (4 pessoas + nenhum dos anteriores)?
- Não funciona bem devido ao tamanho pequeno do treinamento
- Além disso, se um novo funcionário chega, precisa retreinar toda a base (para uma *softmax* de 6 classes)

Função de similaridade

- Em vez disso, aprende-se uma **função de (dis)similaridade**
- Função d entre duas imagens de modo que se essas imagens forem parecidas d vai ter um valor pequeno; se forem bem diferentes, o valor de d será alto
- Na verificação, se d for menor que um *threshold* τ , dizemos que as imagens são da mesma pessoa; Se não, são diferentes
- No reconhecimento, calcula-se d entre a imagem a ser reconhecida e todas as imagens (pessoas) da base

Função de similaridade

- Em vez disso, aprende-se uma **função de (dis)similaridade**
- Função d entre duas imagens de modo que se essas imagens forem parecidas d vai ter um valor pequeno; se forem bem diferentes, o valor de d será alto
- Na verificação, se d for menor que um *threshold* τ , dizemos que as imagens são da mesma pessoa; Se não, são diferentes
- No reconhecimento, calcula-se d entre a imagem a ser reconhecida e todas as imagens (pessoas) da base

Função de similaridade

- Em vez disso, aprende-se uma **função de (dis)similaridade**
- Função d entre duas imagens de modo que se essas imagens forem parecidas d vai ter um valor pequeno; se forem bem diferentes, o valor de d será alto
- Na verificação, se d for menor que um *threshold* τ , dizemos que as imagens são da mesma pessoa; Se não, são diferentes
- No reconhecimento, calcula-se d entre a imagem a ser reconhecida e todas as imagens (pessoas) da base

Função de similaridade

- Em vez disso, aprende-se uma **função de (dis)similaridade**
- Função d entre duas imagens de modo que se essas imagens forem parecidas d vai ter um valor pequeno; se forem bem diferentes, o valor de d será alto
- Na verificação, se d for menor que um *threshold* τ , dizemos que as imagens são da mesma pessoa; Se não, são diferentes
- No reconhecimento, calcula-se d entre a imagem a ser reconhecida e todas as imagens (pessoas) da base

Função de similaridade

- O algoritmo reconhece então como sendo a pessoa na base que teve o menor valor de d
- Para uma imagem que não está na base, espera-se que d atribua um valor alto na comparação com todos da base
- É assim que se resolve o problema do *one shot learning*
- E agora se surge um novo funcionário, basta adicioná-lo na base e tudo vai funcionar bem

Função de similaridade

- O algoritmo reconhece então como sendo a pessoa na base que teve o menor valor de d
- Para uma imagem que não está na base, espera-se que d atribua um valor alto na comparação com todos da base
- É assim que se resolve o problema do *one shot learning*
- E agora se surge um novo funcionário, basta adicioná-lo na base e tudo vai funcionar bem

Função de similaridade

- O algoritmo reconhece então como sendo a pessoa na base que teve o menor valor de d
- Para uma imagem que não está na base, espera-se que d atribua um valor alto na comparação com todos da base
- É assim que se resolve o problema do *one shot learning*
- E agora se surge um novo funcionário, basta adicioná-lo na base e tudo vai funcionar bem

Função de similaridade

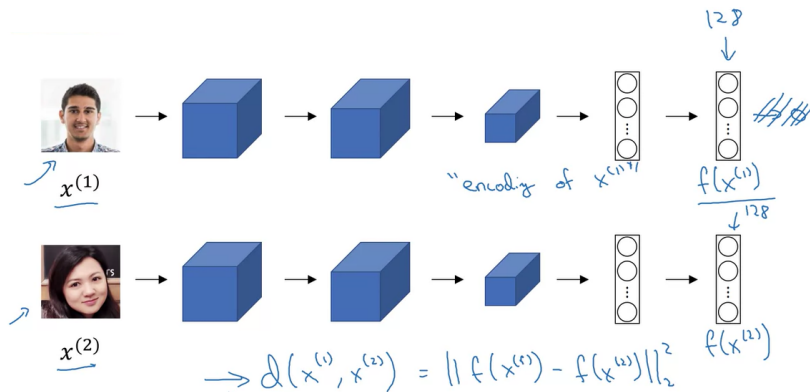
- O algoritmo reconhece então como sendo a pessoa na base que teve o menor valor de d
- Para uma imagem que não está na base, espera-se que d atribua um valor alto na comparação com todos da base
- É assim que se resolve o problema do *one shot learning*
- E agora se surge um novo funcionário, basta adicioná-lo na base e tudo vai funcionar bem

Outline

- 1 O que é Reconhecimento Facial
- 2 One-shot Learning
- 3 Redes Siamesas**
- 4 Verificação Facial e Classificação Binária
- 5 Neural Style Transfer
- 6 Função de Custo
- 7 Convoluções Generalizadas para 1D e 3D

Redes Siamesas¹

- Uma boa forma de aprender a função d é usando uma **rede siamesa**:

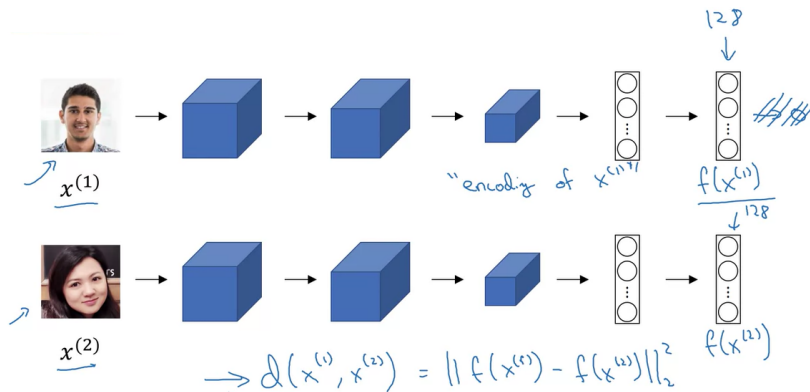


- Paramos agora na penúltima camada FC da ConvNet (p.ex. com 128 unidades), descartando a *softmax*

¹Taigman et al., DeepFace: closing the gap to human level performance

Redes Siamesas¹

- Uma boa forma de aprender a função d é usando uma **rede siamesa**:



- Paramos agora na penúltima camada FC da ConvNet (p.ex. com 128 unidades), descartando a *softmax*

¹Taigman et al., DeepFace: closing the gap to human level performance

Redes Siamesas

- Representamos o vetor com essas 128 saídas para uma entrada $x^{(1)}$ por $f(x^{(1)})$, que será chamado de “*encoding* de $x^{(1)}$ ”
- Para comparar duas imagens $x^{(1)}$ e $x^{(2)}$, usamos ambas como entrada para duas redes idênticas (compartilham os mesmos parâmetros) e observamos os *encodings* $f(x^{(1)})$ e $f(x^{(2)})$

- Considerando que esses *encodings* são boas representações da imagem, a função d é definida então pela norma da diferença entre eles:

$$d = \|f(x^{(1)}) - f(x^{(2)})\|_2^2.$$

- Esse arranjo de duas redes idênticas com entradas diferentes sendo comparadas é o chamado de redes siamesas
- Ideia é apresentada no artigo *DeepFace*, de Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato e Lior Wolf

Redes Siamesas

- Representamos o vetor com essas 128 saídas para uma entrada $x^{(1)}$ por $f(x^{(1)})$, que será chamado de “*encoding* de $x^{(1)}$ ”
- Para comparar duas imagens $x^{(1)}$ e $x^{(2)}$, usamos ambas como entrada para duas redes idênticas (compartilham os mesmos parâmetros) e observamos os *encodings* $f(x^{(1)})$ e $f(x^{(2)})$

- Considerando que esses *encodings* são boas representações da imagem, a função d é definida então pela norma da diferença entre eles:

$$d = \|f(x^{(1)}) - f(x^{(2)})\|_2^2.$$

- Esse arranjo de duas redes idênticas com entradas diferentes sendo comparadas é o chamado de redes siamesas
- Ideia é apresentada no artigo *DeepFace*, de Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato e Lior Wolf

Redes Siamesas

- Representamos o vetor com essas 128 saídas para uma entrada $x^{(1)}$ por $f(x^{(1)})$, que será chamado de “*encoding* de $x^{(1)}$ ”
- Para comparar duas imagens $x^{(1)}$ e $x^{(2)}$, usamos ambas como entrada para duas redes idênticas (compartilham os mesmos parâmetros) e observamos os *encodings* $f(x^{(1)})$ e $f(x^{(2)})$
- Considerando que esses *encodings* são boas representações da imagem, a função d é definida então pela norma da diferença entre eles:

$$d = \|f(x^{(1)}) - f(x^{(2)})\|_2^2.$$

- Esse arranjo de duas redes idênticas com entradas diferentes sendo comparadas é o chamado de redes siamesas
- Ideia é apresentada no artigo *DeepFace*, de Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato e Lior Wolf

Redes Siamesas

- Representamos o vetor com essas 128 saídas para uma entrada $x^{(1)}$ por $f(x^{(1)})$, que será chamado de “*encoding* de $x^{(1)}$ ”
- Para comparar duas imagens $x^{(1)}$ e $x^{(2)}$, usamos ambas como entrada para duas redes idênticas (compartilham os mesmos parâmetros) e observamos os *encodings* $f(x^{(1)})$ e $f(x^{(2)})$
- Considerando que esses *encodings* são boas representações da imagem, a função d é definida então pela norma da diferença entre eles:

$$d = \|f(x^{(1)}) - f(x^{(2)})\|_2^2.$$

- Esse arranjo de duas redes idênticas com entradas diferentes sendo comparadas é o chamado de redes siamesas

- Ideia é apresentada no artigo *DeepFace*, de Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato e Lior Wolf

Redes Siamesas

- Representamos o vetor com essas 128 saídas para uma entrada $x^{(1)}$ por $f(x^{(1)})$, que será chamado de “*encoding* de $x^{(1)}$ ”
- Para comparar duas imagens $x^{(1)}$ e $x^{(2)}$, usamos ambas como entrada para duas redes idênticas (compartilham os mesmos parâmetros) e observamos os *encodings* $f(x^{(1)})$ e $f(x^{(2)})$

- Considerando que esses *encodings* são boas representações da imagem, a função d é definida então pela norma da diferença entre eles:

$$d = \|f(x^{(1)}) - f(x^{(2)})\|_2^2.$$

- Esse arranjo de duas redes idênticas com entradas diferentes sendo comparadas é o chamado de redes siamesas
- Ideia é apresentada no artigo *DeepFace*, de Yaniv Taigman, Ming Yang, Marc'Aurelio Ranzato e Lior Wolf

Redes Siamesas - Treinamento

- Lembre-se de que os parâmetros das duas redes são iguais
- E elas devem ser treinadas de forma que, quando recebem imagens da mesma pessoa, a função d precisa ser pequena
- Mais formalmente, os parâmetros da rede definem o *encoding* $f(x^{(i)})$ e o objetivo é aprender parâmetros tais que:
 - Se $x^{(i)}$ e $x^{(j)}$ são a mesma pessoa, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ é pequeno;
 - Se $x^{(i)}$ e $x^{(j)}$ são pessoas diferentes, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ deve ser grande
- Rodar o backpropagation como de costume para atingir esse objetivo

Redes Siamesas - Treinamento

- Lembre-se de que os parâmetros das duas redes são iguais
- E elas devem ser treinadas de forma que, quando recebem imagens da mesma pessoa, a função d precisa ser pequena
- Mais formalmente, os parâmetros da rede definem o *encoding* $f(x^{(i)})$ e o objetivo é aprender parâmetros tais que:
 - Se $x^{(i)}$ e $x^{(j)}$ são a mesma pessoa, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ é pequeno;
 - Se $x^{(i)}$ e $x^{(j)}$ são pessoas diferentes, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ deve ser grande
- Rodar o backpropagation como de costume para atingir esse objetivo

Redes Siamesas - Treinamento

- Lembre-se de que os parâmetros das duas redes são iguais
- E elas devem ser treinadas de forma que, quando recebem imagens da mesma pessoa, a função d precisa ser pequena
- Mais formalmente, os parâmetros da rede definem o *encoding* $f(x^{(i)})$ e o objetivo é aprender parâmetros tais que:
 - Se $x^{(i)}$ e $x^{(j)}$ são a mesma pessoa, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ é pequeno;
 - Se $x^{(i)}$ e $x^{(j)}$ são pessoas diferentes, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ deve ser grande
- Rodar o backpropagation como de costume para atingir esse objetivo

Redes Siamesas - Treinamento

- Lembre-se de que os parâmetros das duas redes são iguais
- E elas devem ser treinadas de forma que, quando recebem imagens da mesma pessoa, a função d precisa ser pequena
- Mais formalmente, os parâmetros da rede definem o *encoding* $f(x^{(i)})$ e o objetivo é aprender parâmetros tais que:
 - Se $x^{(i)}$ e $x^{(j)}$ são a mesma pessoa, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ é pequeno;
 - Se $x^{(i)}$ e $x^{(j)}$ são pessoas diferentes, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ deve ser grande
- Rodar o backpropagation como de costume para atingir esse objetivo

Redes Siamesas - Treinamento

- Lembre-se de que os parâmetros das duas redes são iguais
- E elas devem ser treinadas de forma que, quando recebem imagens da mesma pessoa, a função d precisa ser pequena
- Mais formalmente, os parâmetros da rede definem o *encoding* $f(x^{(i)})$ e o objetivo é aprender parâmetros tais que:
 - Se $x^{(i)}$ e $x^{(j)}$ são a mesma pessoa, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ é pequeno;
 - Se $x^{(i)}$ e $x^{(j)}$ são pessoas diferentes, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ deve ser grande
- Rodar o backpropagation como de costume para atingir esse objetivo

Redes Siamesas - Treinamento

- Lembre-se de que os parâmetros das duas redes são iguais
- E elas devem ser treinadas de forma que, quando recebem imagens da mesma pessoa, a função d precisa ser pequena
- Mais formalmente, os parâmetros da rede definem o *encoding* $f(x^{(i)})$ e o objetivo é aprender parâmetros tais que:
 - Se $x^{(i)}$ e $x^{(j)}$ são a mesma pessoa, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ é pequeno;
 - Se $x^{(i)}$ e $x^{(j)}$ são pessoas diferentes, então $\|f(x^{(i)}) - f(x^{(j)})\|^2$ deve ser grande
- Rodar o backpropagation como de costume para atingir esse objetivo

Redes Siamesas - Função Objetivo²

- Uma forma de se aprenderem os parâmetros que garantam um bom *encoding* para as faces é usando uma *triplet loss*



Anchor A Positive P

$d(A, P) = 0.5$

Want: $\frac{\|f(A) - f(P)\|^2}{d(A, P)} + \alpha \leq$

$\rightarrow 0.2$

Anchor A Negative N

$d(A, N) = 0.7$

$\frac{\|f(A) - f(N)\|^2}{d(A, N)}$

$$\frac{\|f(A) - f(P)\|^2}{\underbrace{0}} - \frac{\|f(A) - f(N)\|^2}{\underbrace{0}} + \alpha \leq \underbrace{0}_{\text{margin}} \quad \text{if } f(\text{img}) = \vec{0}$$

²Schroff et al., FaceNet: A unified embedding for face recognition and clustering

Redes Siamesas - Função Objetivo

- Essa *loss* vai olhar para pares de imagens
- À esquerda, queremos *encodings* similares porque são as mesmas pessoas; À direita, devem ser bem diferentes porque são pessoas diferentes
- Chamamos as imagens comparadas de âncora (A), positiva (P) (da mesma pessoa) e negativa (N) (de pessoa diferente)
- Daí que vem o nome *triplet loss*, por olhar 3 imagens ao mesmo tempo

Redes Siamesas - Função Objetivo

- Essa *loss* vai olhar para pares de imagens
- À esquerda, queremos *encodings* similares porque são as mesmas pessoas; À direita, devem ser bem diferentes porque são pessoas diferentes
- Chamamos as imagens comparadas de âncora (A), positiva (P) (da mesma pessoa) e negativa (N) (de pessoa diferente)
- Daí que vem o nome *triplet loss*, por olhar 3 imagens ao mesmo tempo

Redes Siamesas - Função Objetivo

- Essa *loss* vai olhar para pares de imagens
- À esquerda, queremos *encodings* similares porque são as mesmas pessoas; À direita, devem ser bem diferentes porque são pessoas diferentes
- Chamamos as imagens comparadas de âncora (A), positiva (P) (da mesma pessoa) e negativa (N) (de pessoa diferente)
- Daí que vem o nome *triplet loss*, por olhar 3 imagens ao mesmo tempo

Redes Siamesas - Função Objetivo

- Essa *loss* vai olhar para pares de imagens
- À esquerda, queremos *encodings* similares porque são as mesmas pessoas; À direita, devem ser bem diferentes porque são pessoas diferentes
- Chamamos as imagens comparadas de âncora (A), positiva (P) (da mesma pessoa) e negativa (N) (de pessoa diferente)
- Daí que vem o nome *triplet loss*, por olhar 3 imagens ao mesmo tempo

Redes Siamesas - Função Objetivo

- Formalizando, queremos $\|f(A) - f(P)\|$ pequeno ou, mais precisamente:

$$d(A, P) = \|f(A) - f(P)\|^2 \leq d(A, N) = \|f(A) - f(N)\|^2,$$

que pode ser rearranjado como

$$\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 \leq 0$$

- Uma forma trivial de satisfazer esse problema é fazendo $f(img) = 0$ para qualquer imagem
- Outra solução trivial seria se o *encoding* de cada duas imagens fosse igual

Redes Siamesas - Função Objetivo

- Formalizando, queremos $\|f(A) - f(P)\|$ pequeno ou, mais precisamente:

$$d(A, P) = \|f(A) - f(P)\|^2 \leq d(A, N) = \|f(A) - f(N)\|^2,$$

que pode ser rearranjado como

$$\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 \leq 0$$

- Uma forma trivial de satisfazer esse problema é fazendo $f(img) = 0$ para qualquer imagem
- Outra solução trivial seria se o *encoding* de cada duas imagens fosse igual

Redes Siamesas - Função Objetivo

- Formalizando, queremos $\|f(A) - f(P)\|$ pequeno ou, mais precisamente:

$$d(A, P) = \|f(A) - f(P)\|^2 \leq d(A, N) = \|f(A) - f(N)\|^2,$$

que pode ser rearranjado como

$$\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 \leq 0$$

- Uma forma trivial de satisfazer esse problema é fazendo $f(img) = 0$ para qualquer imagem
- Outra solução trivial seria se o *encoding* de cada duas imagens fosse igual

Redes Siamesas - Função Objetivo

- Para evitar essa solução trivial, a *loss* precisa ser ajustada
- Isso é feito forçando seu valor ser menor, não que zero, mas sim que um valor $-\alpha$, um pouco menor que zero
- A convenção é escrever:

$$\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha \leq 0$$

Redes Siamesas - Função Objetivo

- Para evitar essa solução trivial, a *loss* precisa ser ajustada
- Isso é feito forçando seu valor ser menor, não que zero, mas sim que um valor $-\alpha$, um pouco menor que zero
- A convenção é escrever:

$$\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha \leq 0$$

Redes Siamesas - Função Objetivo

- Para evitar essa solução trivial, a *loss* precisa ser ajustada
- Isso é feito forçando seu valor ser menor, não que zero, mas sim que um valor $-\alpha$, um pouco menor que zero
- A convenção é escrever:

$$\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha \leq 0$$

Redes Siamesas - Função Objetivo

- α é um hiperparâmetro do modelo e é chamado de margem
- α pode também ser adicionado na equação de cima
- Assim, se p.ex. $d(A, P) = 0.5$ e $\alpha = 0.2$, não basta que $d(A, N) > 0.5$, p.ex., 0.51, mas precisa ser ainda maior, acima de 0.7
- Isso distancia os pares AN e AP

Redes Siamesas - Função Objetivo

- α é um hiperparâmetro do modelo e é chamado de margem
- α pode também ser adicionado na equação de cima
- Assim, se p.ex. $d(A, P) = 0.5$ e $\alpha = 0.2$, não basta que $d(A, N) > 0.5$, p.ex., 0.51, mas precisa ser ainda maior, acima de 0.7
- Isso distancia os pares AN e AP

Redes Siamesas - Função Objetivo

- α é um hiperparâmetro do modelo e é chamado de margem
- α pode também ser adicionado na equação de cima
- Assim, se p.ex. $d(A, P) = 0.5$ e $\alpha = 0.2$, não basta que $d(A, N) > 0.5$, p.ex., 0.51, mas precisa ser ainda maior, acima de 0.7
- Isso distancia os pares AN e AP

Redes Siamesas - Função Objetivo

- α é um hiperparâmetro do modelo e é chamado de margem
- α pode também ser adicionado na equação de cima
- Assim, se p.ex. $d(A, P) = 0.5$ e $\alpha = 0.2$, não basta que $d(A, N) > 0.5$, p.ex., 0.51, mas precisa ser ainda maior, acima de 0.7
- Isso distancia os pares AN e AP

Redes Siamesas - Função Objetivo (Definição)

- Dadas 3 imagens A (âncora), P (positiva) e N (negativa), em que A e P são da mesma pessoa, mas A e N são de pessoas diferentes, então a *triplet loss* é dada por

$$\mathcal{L}(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

- Se o primeiro argumento do max for negativo, então a *loss* é zero
- Se for positivo, a rede vai tentar minimizar essa parcela
- O ponto é que, se essa parcela for menor que zero, não importa o quão negativa ela é

Redes Siamesas - Função Objetivo (Definição)

- Dadas 3 imagens A (âncora), P (positiva) e N (negativa), em que A e P são da mesma pessoa, mas A e N são de pessoas diferentes, então a *triplet loss* é dada por

$$\mathcal{L}(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

- Se o primeiro argumento do max for negativo, então a *loss* é zero
- Se for positivo, a rede vai tentar minimizar essa parcela
- O ponto é que, se essa parcela for menor que zero, não importa o quão negativa ela é

Redes Siamesas - Função Objetivo (Definição)

- Dadas 3 imagens A (âncora), P (positiva) e N (negativa), em que A e P são da mesma pessoa, mas A e N são de pessoas diferentes, então a *triplet loss* é dada por

$$\mathcal{L}(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

- Se o primeiro argumento do max for negativo, então a *loss* é zero
- Se for positivo, a rede vai tentar minimizar essa parcela
- O ponto é que, se essa parcela for menor que zero, não importa o quão negativa ela é

Redes Siamesas - Função Objetivo (Definição)

- Dadas 3 imagens A (âncora), P (positiva) e N (negativa), em que A e P são da mesma pessoa, mas A e N são de pessoas diferentes, então a *triplet loss* é dada por

$$\mathcal{L}(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

- Se o primeiro argumento do max for negativo, então a *loss* é zero
- Se for positivo, a rede vai tentar minimizar essa parcela
- O ponto é que, se essa parcela for menor que zero, não importa o quão negativa ela é

Redes Siamesas - Função Objetivo (Definição)

- Essa é a *loss* para uma única *triplet*; A função de custo é, como de costume, a soma sobre todas as *triplets* no treinamento:

$$J = \sum_{i=1}^m \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)}).$$

- Suponha uma base com 10 mil imagens de mil pessoas
- Deve-se usar então essas 10 mil imagens para formar as *triplets* e então aplicar o gradiente descendente sobre a *loss* acima
- Note que no exemplo acima temos 10 mil imagens de mil pessoas para montar os pares AP; Se tivéssemos uma imagem apenas de cada pessoa, não teríamos como treinar

Redes Siamesas - Função Objetivo (Definição)

- Essa é a *loss* para uma única *triplet*; A função de custo é, como de costume, a soma sobre todas as *triplets* no treinamento:

$$J = \sum_{i=1}^m \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)}).$$

- Suponha uma base com 10 mil imagens de mil pessoas
- Deve-se usar então essas 10 mil imagens para formar as *triplets* e então aplicar o gradiente descendente sobre a *loss* acima
- Note que no exemplo acima temos 10 mil imagens de mil pessoas para montar os pares AP; Se tivéssemos uma imagem apenas de cada pessoa, não teríamos como treinar

Redes Siamesas - Função Objetivo (Definição)

- Essa é a *loss* para uma única *triplet*; A função de custo é, como de costume, a soma sobre todas as *triplets* no treinamento:

$$J = \sum_{i=1}^m \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)}).$$

- Suponha uma base com 10 mil imagens de mil pessoas
- Deve-se usar então essas 10 mil imagens para formar as *triplets* e então aplicar o gradiente descendente sobre a *loss* acima
- Note que no exemplo acima temos 10 mil imagens de mil pessoas para montar os pares AP; Se tivéssemos uma imagem apenas de cada pessoa, não teríamos como treinar

Redes Siamesas - Função Objetivo (Definição)

- Essa é a *loss* para uma única *triplet*; A função de custo é, como de costume, a soma sobre todas as *triplets* no treinamento:

$$J = \sum_{i=1}^m \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)}).$$

- Suponha uma base com 10 mil imagens de mil pessoas
- Deve-se usar então essas 10 mil imagens para formar as *triplets* e então aplicar o gradiente descendente sobre a *loss* acima
- Note que no exemplo acima temos 10 mil imagens de mil pessoas para montar os pares AP; Se tivéssemos uma imagem apenas de cada pessoa, não teríamos como treinar

Redes Siamesas - Treinamento

- Porém, com o sistema já treinado, aí bastará uma única imagem da pessoa para que se faça o reconhecimento e se resolva o problema de *one-shot learning*
- Mas como escolher efetivamente essas triplets?

Anchor



⋮



Positive



⋮



Negative



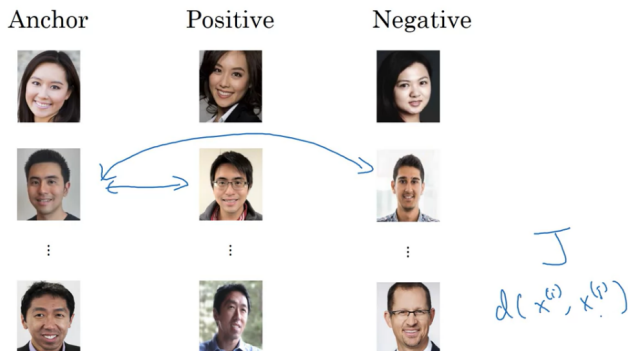
⋮



$$J = d(x^{(a)}, x^{(p)})$$

Redes Siamesas - Treinamento

- Porém, com o sistema já treinado, aí bastará uma única imagem da pessoa para que se faça o reconhecimento e se resolva o problema de *one-shot learning*
- Mas como escolher efetivamente essas triplets?



Redes Siamesas - Treinamento

- O problema de escolher A , P e N aleatoriamente é que a restrição $d(A, P) + \alpha \leq d(A, N)$ é facilmente satisfeita, ou seja:

$$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2.$$

- Assim, a rede não aprenderia muito com esses dados
- O objetivo é escolher *triplets* mais “difíceis” para treinar
- Devemos satisfazer então, ao mesmo tempo:

$$d(A, P) + \alpha \leq d(A, N)$$

$$d(A, P) \approx \leq d(A, N)$$

- Isso é atingido diminuindo $d(A, P)$ e aumentando $d(A, N)$

Redes Siamesas - Treinamento

- O problema de escolher A , P e N aleatoriamente é que a restrição $d(A, P) + \alpha \leq d(A, N)$ é facilmente satisfeita, ou seja:

$$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2.$$

- Assim, a rede não aprenderia muito com esses dados
- O objetivo é escolher *triplets* mais “difíceis” para treinar
- Devemos satisfazer então, ao mesmo tempo:

$$d(A, P) + \alpha \leq d(A, N)$$

$$d(A, P) \approx \leq d(A, N)$$

- Isso é atingido diminuindo $d(A, P)$ e aumentando $d(A, N)$

Redes Siamesas - Treinamento

- O problema de escolher A , P e N aleatoriamente é que a restrição $d(A, P) + \alpha \leq d(A, N)$ é facilmente satisfeita, ou seja:

$$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2.$$

- Assim, a rede não aprenderia muito com esses dados
- O objetivo é escolher *triplets* mais “difíceis” para treinar
- Devemos satisfazer então, ao mesmo tempo:

$$d(A, P) + \alpha \leq d(A, N)$$

$$d(A, P) \approx \leq d(A, N)$$

- Isso é atingido diminuindo $d(A, P)$ e aumentando $d(A, N)$

Redes Siamesas - Treinamento

- O problema de escolher A , P e N aleatoriamente é que a restrição $d(A, P) + \alpha \leq d(A, N)$ é facilmente satisfeita, ou seja:

$$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2.$$

- Assim, a rede não aprenderia muito com esses dados
- O objetivo é escolher *triplets* mais “difíceis” para treinar
- Devemos satisfazer então, ao mesmo tempo:

$$d(A, P) + \alpha \leq d(A, N)$$

$$d(A, P) \approx \leq d(A, N)$$

- Isso é atingido diminuindo $d(A, P)$ e aumentando $d(A, N)$

Redes Siamesas - Treinamento

- O problema de escolher A , P e N aleatoriamente é que a restrição $d(A, P) + \alpha \leq d(A, N)$ é facilmente satisfeita, ou seja:

$$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2.$$

- Assim, a rede não aprenderia muito com esses dados
- O objetivo é escolher *triplets* mais “difíceis” para treinar
- Devemos satisfazer então, ao mesmo tempo:

$$d(A, P) + \alpha \leq d(A, N)$$

$$d(A, P) \approx \leq d(A, N)$$

- Isso é atingido diminuindo $d(A, P)$ e aumentando $d(A, N)$

Redes Siamesas - Treinamento

- Detalhes são fornecidos no artigo FaceNet de Florian Schroff, Dmitry Kalenichenko e James Philbin
- CURIOSIDADE: Essa ideia de dar nome a algoritmos de *deep learning* como “algo”Net ou Deep “algo” (ex. FaceNet e DeepFace) é muito comum
- Em resumo, vamos treinar o algoritmo com as *triplets* A,P,N, como nos exemplos abaixo, e minimizar a função de custo que vimos usando *backpropagation*
- Isso visando aprender um *encoding* $d(x^{(i)}, x^{(j)})$ que gera um valor pequeno para imagens da mesma pessoa e grande para pessoas diferentes

Redes Siamesas - Treinamento

- Detalhes são fornecidos no artigo FaceNet de Florian Schroff, Dmitry Kalenichenko e James Philbin
- CURIOSIDADE: Essa ideia de dar nome a algoritmos de *deep learning* como “algo” Net ou Deep “algo” (ex. FaceNet e DeepFace) é muito comum
- Em resumo, vamos treinar o algoritmo com as *triplets* A,P,N, como nos exemplos abaixo, e minimizar a função de custo que vimos usando *backpropagation*
- Isso visando aprender um *encoding* $d(x^{(i)}, x^{(j)})$ que gera um valor pequeno para imagens da mesma pessoa e grande para pessoas diferentes

Redes Siamesas - Treinamento

- Detalhes são fornecidos no artigo FaceNet de Florian Schroff, Dmitry Kalenichenko e James Philbin
- CURIOSIDADE: Essa ideia de dar nome a algoritmos de *deep learning* como “algo” Net ou Deep “algo” (ex. FaceNet e DeepFace) é muito comum
- Em resumo, vamos treinar o algoritmo com as *triplets* A,P,N, como nos exemplos abaixo, e minimizar a função de custo que vimos usando *backpropagation*
- Isso visando aprender um *encoding* $d(x^{(i)}, x^{(j)})$ que gera um valor pequeno para imagens da mesma pessoa e grande para pessoas diferentes

Redes Siamesas - Treinamento

- Detalhes são fornecidos no artigo FaceNet de Florian Schroff, Dmitry Kalenichenko e James Philbin
- CURIOSIDADE: Essa ideia de dar nome a algoritmos de *deep learning* como “algo” Net ou Deep “algo” (ex. FaceNet e DeepFace) é muito comum
- Em resumo, vamos treinar o algoritmo com as *triplets* A,P,N, como nos exemplos abaixo, e minimizar a função de custo que vimos usando *backpropagation*
- Isso visando aprender um *encoding* $d(x^{(i)}, x^{(j)})$ que gera um valor pequeno para imagens da mesma pessoa e grande para pessoas diferentes

Redes Siamesas - Treinamento

- NOTA: Os sistemas atuais de reconhecimento facial, sobretudo os comerciais, são treinados sobre bases muito grandes, da ordem de muitos milhões de imagens
- Felizmente, muitos grandes projetos têm treinado modelos com essas bases e disponibilizado os parâmetros online
- Este é portanto um domínio em que o uso de modelos pré-treinados é muito conveniente

Redes Siamesas - Treinamento

- NOTA: Os sistemas atuais de reconhecimento facial, sobretudo os comerciais, são treinados sobre bases muito grandes, da ordem de muitos milhões de imagens
- Felizmente, muitos grandes projetos têm treinado modelos com essas bases e disponibilizado os parâmetros online
- Este é portanto um domínio em que o uso de modelos pré-treinados é muito conveniente

Redes Siamesas - Treinamento

- NOTA: Os sistemas atuais de reconhecimento facial, sobretudo os comerciais, são treinados sobre bases muito grandes, da ordem de muitos milhões de imagens
- Felizmente, muitos grandes projetos têm treinado modelos com essas bases e disponibilizado os parâmetros online
- Este é portanto um domínio em que o uso de modelos pré-treinados é muito conveniente

Outline

- 1 O que é Reconhecimento Facial
- 2 One-shot Learning
- 3 Redes Siamesas
- 4 Verificação Facial e Classificação Binária**
- 5 Neural Style Transfer
- 6 Função de Custo
- 7 Convoluções Generalizadas para 1D e 3D

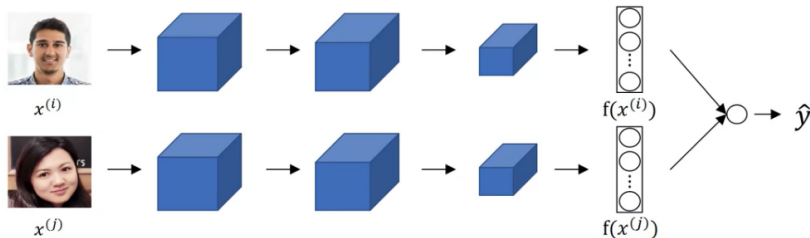
Classificação Binária³

- Embora a *triplet loss* seja uma ótima forma de se aprenderem parâmetros em reconhecimento facial, existem outras soluções
- Uma delas é tratar o reconhecimento facial como um problema de classificação binária

³Taigman et al., DeepFace: closing the gap to human level performance

Classificação Binária³

- Embora a *triplet loss* seja uma ótima forma de se aprenderem parâmetros em reconhecimento facial, existem outras soluções
- Uma delas é tratar o reconhecimento facial como um problema de classificação binária



³Taigman et al., DeepFace: closing the gap to human level performance

Classificação Binária

- Continuamos aprendendo uma função de similaridade, porém, a entrada agora é um par de imagens e a saída é 0 ou 1 dependendo de as imagens serem de pessoas iguais ou diferentes
- Usamos novamente uma rede siamesa, mas dessa vez para calcular $f(x^{(i)})$ e $f(x^{(j)})$
- Isso é chamado também de *embedding* e pode ter dimensão, p.ex., 128 ou mesmo maior
- Correspondem aos *features* da imagem e são usados como entrada de uma regressão logística, de modo que a saída seja 1 para pares da mesma pessoa e 0 para pessoas diferentes

Classificação Binária

- Continuamos aprendendo uma função de similaridade, porém, a entrada agora é um par de imagens e a saída é 0 ou 1 dependendo de as imagens serem de pessoas iguais ou diferentes
- Usamos novamente uma rede siamesa, mas dessa vez para calcular $f(x^{(i)})$ e $f(x^{(j)})$
- Isso é chamado também de *embedding* e pode ter dimensão, p.ex., 128 ou mesmo maior
- Correspondem aos *features* da imagem e são usados como entrada de uma regressão logística, de modo que a saída seja 1 para pares da mesma pessoa e 0 para pessoas diferentes

Classificação Binária

- Continuamos aprendendo uma função de similaridade, porém, a entrada agora é um par de imagens e a saída é 0 ou 1 dependendo de as imagens serem de pessoas iguais ou diferentes
- Usamos novamente uma rede siamesa, mas dessa vez para calcular $f(x^{(i)})$ e $f(x^{(j)})$
- Isso é chamado também de *embedding* e pode ter dimensão, p.ex., 128 ou mesmo maior
- Correspondem aos *features* da imagem e são usados como entrada de uma regressão logística, de modo que a saída seja 1 para pares da mesma pessoa e 0 para pessoas diferentes

Classificação Binária

- Continuamos aprendendo uma função de similaridade, porém, a entrada agora é um par de imagens e a saída é 0 ou 1 dependendo de as imagens serem de pessoas iguais ou diferentes
- Usamos novamente uma rede siamesa, mas dessa vez para calcular $f(x^{(i)})$ e $f(x^{(j)})$
- Isso é chamado também de *embedding* e pode ter dimensão, p.ex., 128 ou mesmo maior
- Correspondem aos *features* da imagem e são usados como entrada de uma regressão logística, de modo que a saída seja 1 para pares da mesma pessoa e 0 para pessoas diferentes

Classificação Binária

- No caso, a entrada para a sigmoide logística vai ser a diferença componente a componente entre os *embeddings*:

$$y = \sigma \left(\sum_{k=1}^{128} w_k |f(x^{(i)})_k - f(x^{(j)})_k| + b \right),$$

em que $f(x^{(i)})_k$ é o k -ésimo componente do *encoding*

- Podem-se usar ainda outras funções nessa entrada, como a dissimilaridade chi-quadrado (χ^2):

$$y = \sigma \left(\sum_{k=1}^{128} w_k \frac{(f(x^{(i)})_k - f(x^{(j)})_k)^2}{f(x^{(i)})_k + f(x^{(j)})_k} + b \right),$$

- O treinamento é idêntico ao que já conhecemos, novamente com as redes “amarradas”, tendo a rede de cima os mesmos parâmetros da rede de baixo

Classificação Binária

- No caso, a entrada para a sigmoide logística vai ser a diferença componente a componente entre os *embeddings*:

$$y = \sigma \left(\sum_{k=1}^{128} w_k |f(x^{(i)})_k - f(x^{(j)})_k| + b \right),$$

em que $f(x^{(i)})_k$ é o k -ésimo componente do *encoding*

- Podem-se usar ainda outras funções nessa entrada, como a dissimilaridade chi-quadrado (χ^2):

$$y = \sigma \left(\sum_{k=1}^{128} w_k \frac{(f(x^{(i)})_k - f(x^{(j)})_k)^2}{f(x^{(i)})_k + f(x^{(j)})_k} + b \right),$$

- O treinamento é idêntico ao que já conhecemos, novamente com as redes “amarradas”, tendo a rede de cima os mesmos parâmetros da rede de baixo

Classificação Binária

- No caso, a entrada para a sigmoide logística vai ser a diferença componente a componente entre os *embeddings*:

$$y = \sigma \left(\sum_{k=1}^{128} w_k |f(x^{(i)})_k - f(x^{(j)})_k| + b \right),$$

em que $f(x^{(i)})_k$ é o k -ésimo componente do *encoding*

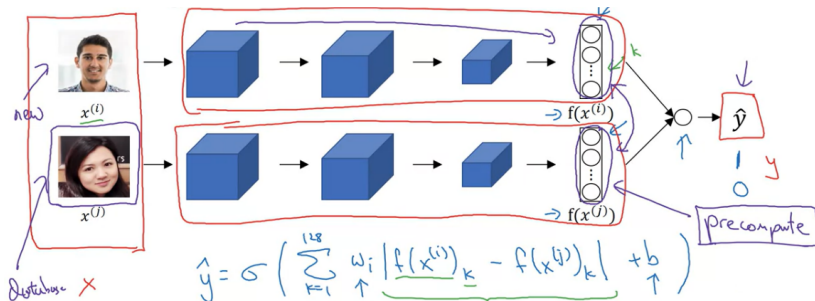
- Podem-se usar ainda outras funções nessa entrada, como a dissimilaridade chi-quadrado (χ^2):

$$y = \sigma \left(\sum_{k=1}^{128} w_k \frac{(f(x^{(i)})_k - f(x^{(j)})_k)^2}{f(x^{(i)})_k + f(x^{(j)})_k} + b \right),$$

- O treinamento é idêntico ao que já conhecemos, novamente com as redes “amarradas”, tendo a rede de cima os mesmos parâmetros da rede de baixo

Classificação Binária

- Um truque computacional importante é que se você vai usar um par que inclui um funcionário já conhecido na sua base e outro novo, o *embedding* do conhecido já pode ser pré-calculado e salvo, treinando-se a rede siamesa apenas sobre a imagem do funcionário novo e o *embedding* do que já é conhecido na base



Classificação Binária

- Assim tanto não precisa ficar calculando várias vezes o mesmo *encoding* quanto também não é necessário salvar toda a imagem, mas apenas o *encoding*
- Essa ideia também funciona na *triplet loss*

Classificação Binária

- Assim tanto não precisa ficar calculando várias vezes o mesmo *encoding* quanto também não é necessário salvar toda a imagem, mas apenas o *encoding*
- Essa ideia também funciona na *triplet loss*

Classificação Binária

- A figura abaixo resume o processo supervisionado para verificação (e consequentemente reconhecimento) facial:

x	y	
		1 "Same"
		0 "Different"
		0
		1

Outline

- 1 O que é Reconhecimento Facial
- 2 One-shot Learning
- 3 Redes Siamesas
- 4 Verificação Facial e Classificação Binária
- 5 Neural Style Transfer**
- 6 Função de Custo
- 7 Convoluções Generalizadas para 1D e 3D

O que é Neural Style Transfer

- Uma das aplicações mais curiosas das ConvNets
- Você pode sem nenhuma habilidade artística gerar as suas obras de arte⁴

⁴Imagens geradas por Justin Johnson

O que é Neural Style Transfer

- Uma das aplicações mais curiosas das ConvNets
- Você pode sem nenhuma habilidade artística gerar as suas obras de arte⁴

Neural style transfer

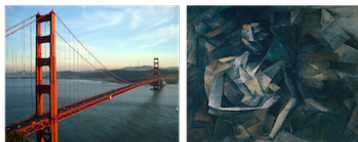


Content (C)

Style (S)

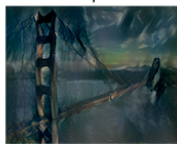


Generated image (G)



Content (C)

Style (S)



Generated image (G)

⁴Imagens geradas por Justin Johnson

O que é Neural Style Transfer

- Temos aí uma foto da Universidade de Stanford e a pintura “Noite Estrelada” de Van Gogh
- O *neural style transfer* gera uma imagem da universidade no estilo da pintura de Van Gogh
- Chamamos a imagem original de “Conteúdo” (C), a imagem com o estilo de “Estilo” (S) e a imagem resultante de “Gerada” (G)
- O outro exemplo é a ponte Golden Gate em San Francisco com estilo de Pablo Picaso
- Para isso, vamos olhar para os *features* extraídos por uma ConvNet em diferentes camadas

O que é Neural Style Transfer

- Temos aí uma foto da Universidade de Stanford e a pintura “Noite Estrelada” de Van Gogh
- O *neural style transfer* gera uma imagem da universidade no estilo da pintura de Van Gogh
- Chamamos a imagem original de “Conteúdo” (C), a imagem com o estilo de “Estilo” (S) e a imagem resultante de “Gerada” (G)
- O outro exemplo é a ponte Golden Gate em San Francisco com estilo de Pablo Picaso
- Para isso, vamos olhar para os *features* extraídos por uma ConvNet em diferentes camadas

O que é Neural Style Transfer

- Temos aí uma foto da Universidade de Stanford e a pintura “Noite Estrelada” de Van Gogh
- O *neural style transfer* gera uma imagem da universidade no estilo da pintura de Van Gogh
- Chamamos a imagem original de “Conteúdo” (C), a imagem com o estilo de “Estilo” (S) e a imagem resultante de “Gerada” (G)
- O outro exemplo é a ponte Golden Gate em San Francisco com estilo de Pablo Picaso
- Para isso, vamos olhar para os *features* extraídos por uma ConvNet em diferentes camadas

O que é Neural Style Transfer

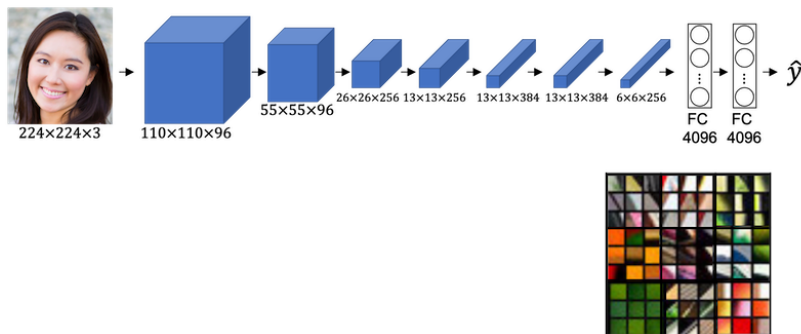
- Temos aí uma foto da Universidade de Stanford e a pintura “Noite Estrelada” de Van Gogh
- O *neural style transfer* gera uma imagem da universidade no estilo da pintura de Van Gogh
- Chamamos a imagem original de “Conteúdo” (C), a imagem com o estilo de “Estilo” (S) e a imagem resultante de “Gerada” (G)
- O outro exemplo é a ponte Golden Gate em San Francisco com estilo de Pablo Picaso
- Para isso, vamos olhar para os *features* extraídos por uma ConvNet em diferentes camadas

O que é Neural Style Transfer

- Temos aí uma foto da Universidade de Stanford e a pintura “Noite Estrelada” de Van Gogh
- O *neural style transfer* gera uma imagem da universidade no estilo da pintura de Van Gogh
- Chamamos a imagem original de “Conteúdo” (C), a imagem com o estilo de “Estilo” (S) e a imagem resultante de “Gerada” (G)
- O outro exemplo é a ponte Golden Gate em San Francisco com estilo de Pablo Picaso
- Para isso, vamos olhar para os *features* extraídos por uma ConvNet em diferentes camadas

O que as ConvNets Estão Aprendendo?⁵

- Imagine a ConvNet abaixo, similar à AlexNet



- Suponha uma unidade escondida na camada 1
- Passamos as imagens de treinamento pela rede e identificar qual imagem ou qual *patch* maximiza a ativação daquela unidade

⁵Zeiler and Fergus, 2013, Visualizing and understanding convolutional networks

O que as ConvNets Estão Aprendendo?⁵

- Imagine a ConvNet abaixo, similar à AlexNet



- Suponha uma unidade escondida na camada 1
- Passamos as imagens de treinamento pela rede e identificar qual imagem ou qual *patch* maximiza a ativação daquela unidade

⁵Zeiler and Fergus, 2013, Visualizing and understanding convolutional networks

O que as ConvNets Estão Aprendendo?⁵

- Imagine a ConvNet abaixo, similar à AlexNet



- Suponha uma unidade escondida na camada 1
- Passamos as imagens de treinamento pela rede e identificar qual imagem ou qual patch maximiza a ativação daquela unidade

⁵Zeiler and Fergus, 2013, Visualizing and understanding convolutional networks

O que as ConvNets Estão Aprendendo?

- Note que cada unidade escondida olha apenas para uma pequena parte da entrada (campo receptivo)
- Então a parte que ativa aquela unidade é apenas um pequeno *patch* da imagem
- No exemplo, cada unidade escondida na camada 1 olha para 9 *patches* e cada um desses *patches* corresponde a um *feature* primitivo como borda ou linha
- Da mesma forma, outras unidades escondidas vão olhar para outros 9 *patches*
- Temos, por exemplo, bordas mais leves e com preferência para a cor verde à esquerda, outro já foca no laranja, outros em uma mistura de verde e vermelho, etc.

O que as ConvNets Estão Aprendendo?

- Note que cada unidade escondida olha apenas para uma pequena parte da entrada (campo receptivo)
- Então a parte que ativa aquela unidade é apenas um pequeno *patch* da imagem
- No exemplo, cada unidade escondida na camada 1 olha para 9 *patches* e cada um desses *patches* corresponde a um *feature* primitivo como borda ou linha
- Da mesma forma, outras unidades escondidas vão olhar para outros 9 *patches*
- Temos, por exemplo, bordas mais leves e com preferência para a cor verde à esquerda, outro já foca no laranja, outros em uma mistura de verde e vermelho, etc.

O que as ConvNets Estão Aprendendo?

- Note que cada unidade escondida olha apenas para uma pequena parte da entrada (campo receptivo)
- Então a parte que ativa aquela unidade é apenas um pequeno *patch* da imagem
- No exemplo, cada unidade escondida na camada 1 olha para 9 *patches* e cada um desses *patches* corresponde a um *feature* primitivo como borda ou linha
- Da mesma forma, outras unidades escondidas vão olhar para outros 9 *patches*
- Temos, por exemplo, bordas mais leves e com preferência para a cor verde à esquerda, outro já foca no laranja, outros em uma mistura de verde e vermelho, etc.

O que as ConvNets Estão Aprendendo?

- Note que cada unidade escondida olha apenas para uma pequena parte da entrada (campo receptivo)
- Então a parte que ativa aquela unidade é apenas um pequeno *patch* da imagem
- No exemplo, cada unidade escondida na camada 1 olha para 9 *patches* e cada um desses *patches* corresponde a um *feature* primitivo como borda ou linha
- Da mesma forma, outras unidades escondidas vão olhar para outros 9 *patches*
- Temos, por exemplo, bordas mais leves e com preferência para a cor verde à esquerda, outro já foca no laranja, outros em uma mistura de verde e vermelho, etc.

O que as ConvNets Estão Aprendendo?

- Note que cada unidade escondida olha apenas para uma pequena parte da entrada (campo receptivo)
- Então a parte que ativa aquela unidade é apenas um pequeno *patch* da imagem
- No exemplo, cada unidade escondida na camada 1 olha para 9 *patches* e cada um desses *patches* corresponde a um *feature* primitivo como borda ou linha
- Da mesma forma, outras unidades escondidas vão olhar para outros 9 *patches*
- Temos, por exemplo, bordas mais leves e com preferência para a cor verde à esquerda, outro já foca no laranja, outros em uma mistura de verde e vermelho, etc.

O que as ConvNets Estão Aprendendo?

- Todos esses exemplos vêm do artigo de Mathew Zeiler e Rob Fergus, “Visualizing and understanding convolutional networks”
- Lá eles também apresentam formas mais sofisticadas de se visualizar uma ConvNet
- E quando vamos para camadas mais profundas? As unidades vão olhar agora para regiões maiores da imagem
- Vemos também os 9 *patches* que causam uma alta ativação em 9 unidades escondidas da camada 2. Mesma coisa para camadas mais profundas

O que as ConvNets Estão Aprendendo?

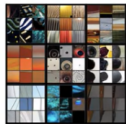
- Todos esses exemplos vêm do artigo de Mathew Zeiler e Rob Fergus, “Visualizing and understanding convolutional networks”
- Lá eles também apresentam formas mais sofisticadas de se visualizar uma ConvNet
- E quando vamos para camadas mais profundas? As unidades vão olhar agora para regiões maiores da imagem
- Vemos também os 9 *patches* que causam uma alta ativação em 9 unidades escondidas da camada 2. Mesma coisa para camadas mais profundas

O que as ConvNets Estão Aprendendo?

- Todos esses exemplos vêm do artigo de Mathew Zeiler e Rob Fergus, “Visualizing and understanding convolutional networks”
- Lá eles também apresentam formas mais sofisticadas de se visualizar uma ConvNet
- E quando vamos para camadas mais profundas? As unidades vão olhar agora para regiões maiores da imagem



Layer 1



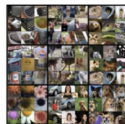
Layer 2



Layer 3



Layer 4



Layer 5

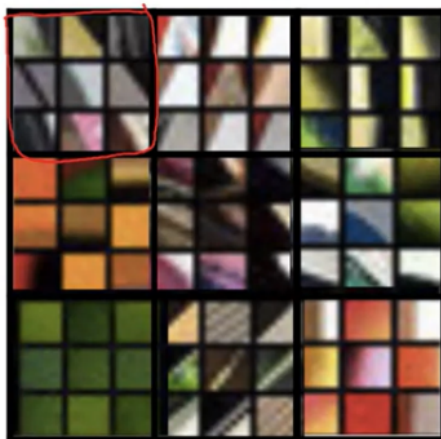
- Vemos também os 9 *patches* que causam uma alta ativação em 9 unidades escondidas da camada 2. Mesma coisa para camadas mais profundas

O que as ConvNets Estão Aprendendo?

- Todos esses exemplos vêm do artigo de Mathew Zeiler e Rob Fergus, “Visualizing and understanding convolutional networks”
- Lá eles também apresentam formas mais sofisticadas de se visualizar uma ConvNet
- E quando vamos para camadas mais profundas? As unidades vão olhar agora para regiões maiores da imagem
- Vemos também os 9 *patches* que causam uma alta ativação em 9 unidades escondidas da camada 2. Mesma coisa para camadas mais profundas

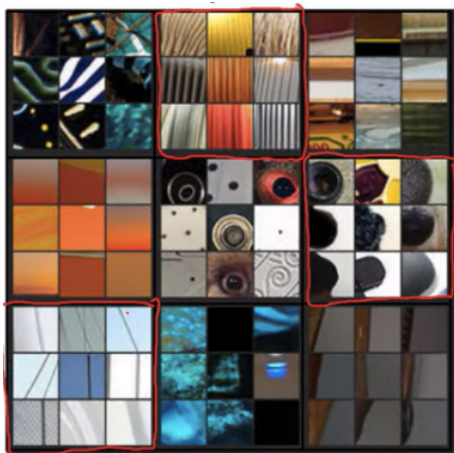
O que as ConvNets Estão Aprendendo?

- Podemos agora dar um zoom na camada 1, onde observamos bordas em diferentes direções:



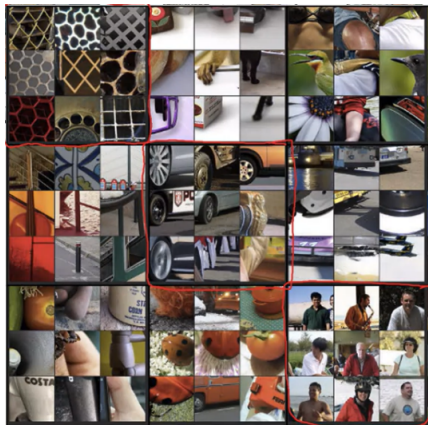
O que as ConvNets Estão Aprendendo?

- Na camada 2, observamos *features* mais complexos como texturas verticais, objetos arredondados à esquerda, linhas verticais finas, etc.



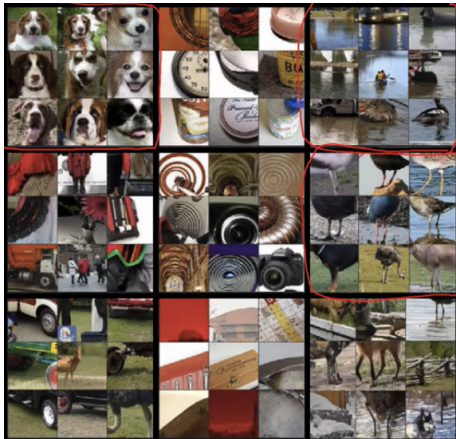
O que as ConvNets Estão Aprendendo?

- Na camada 3, são detectados carros (objetos no canto inferior esquerdo), pessoas, texturas em forma de favo, quadradas e irregulares, etc; Tudo claramente mais complexo do que a camada anterior



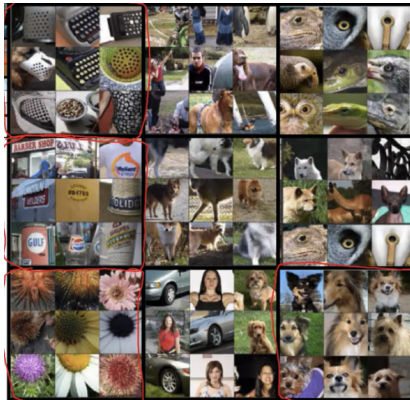
O que as ConvNets Estão Aprendendo?

- Na camada 4, estamos detectando cachorros (e note que todos muito parecidos), água, patas de um pássaro, etc.



O que as ConvNets Estão Aprendendo?

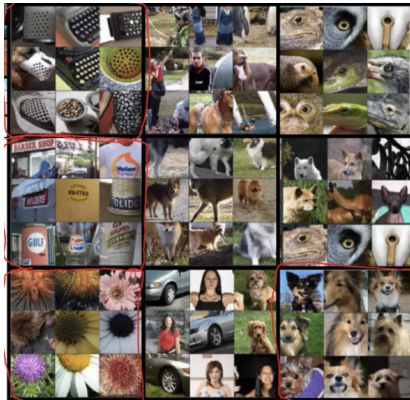
- Na camada 5, detectamos cachorros, mas variados, teclados (pontos em um fundo), texto e flores



- Note como partimos da camada 1 detectando bordas, detectamos texturas na camada 2, até chegar em objetos complexos nas camadas mais profundas

O que as ConvNets Estão Aprendendo?

- Na camada 5, detectamos cachorros, mas variados, teclados (pontos em um fundo), texto e flores



- Note como partimos da camada 1 detectando bordas, detectamos texturas na camada 2, até chegar em objetos complexos nas camadas mais profundas

Outline

- 1 O que é Reconhecimento Facial
- 2 One-shot Learning
- 3 Redes Siamesas
- 4 Verificação Facial e Classificação Binária
- 5 Neural Style Transfer
- 6 Função de Custo**
- 7 Convoluções Generalizadas para 1D e 3D

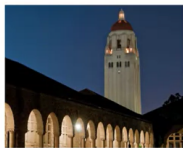
Função de Custo⁶

- Lembrando da formulação do problema: dada uma imagem de conteúdo (C) e de estilo (S), o objetivo é obter a imagem gerada (G)
- Definimos uma função de custo J , que medirá quão boa é a imagem G e usaremos gradiente descendente para minimizar J

⁶Gatys et al., 2015. A neural algorithm of artistic style

Função de Custo⁶

- Lembrando da formulação do problema: dada uma imagem de conteúdo (C) e de estilo (S), o objetivo é obter a imagem gerada (G)
- Definimos uma função de custo J , que medirá quão boa é a imagem G e usaremos gradiente descendente para minimizar J



Content C



Style S



Generated image G

$$J(G) = \alpha J_{\text{content}}(C, G) + \beta J_{\text{style}}(S, G)$$

⁶Gatys et al., 2015. A neural algorithm of artistic style

Função de Custo

- J será formada por duas partes: $J_{content}$, que mede o quanto C e G são similares e J_{style} , que deve medir o quanto o estilo das imagens G e S são similares
- Por fim, as duas funções são ponderadas por hiperparâmetros α e β
- NOTA: Pode parecer redundante usar dois hiperparâmetros para dar pesos relativos, mas essa é a convenção no artigo original
- NOTA 2: Os autores são Leon Gatys, Alexander Ecker e Matthias; Não é um artigo difícil de ler

Função de Custo

- J será formada por duas partes: $J_{content}$, que mede o quanto C e G são similares e J_{style} , que deve medir o quanto o estilo das imagens G e S são similares
- Por fim, as duas funções são ponderadas por hiperparâmetros α e β
- NOTA: Pode parecer redundante usar dois hiperparâmetros para dar pesos relativos, mas essa é a convenção no artigo original
- NOTA 2: Os autores são Leon Gatys, Alexander Ecker e Matthias; Não é um artigo difícil de ler

Função de Custo

- J será formada por duas partes: $J_{content}$, que mede o quanto C e G são similares e J_{style} , que deve medir o quanto o estilo das imagens G e S são similares
- Por fim, as duas funções são ponderadas por hiperparâmetros α e β
- NOTA: Pode parecer redundante usar dois hiperparâmetros para dar pesos relativos, mas essa é a convenção no artigo original
- NOTA 2: Os autores são Leon Gatys, Alexander Ecker e Matthias; Não é um artigo difícil de ler

Função de Custo

- J será formada por duas partes: $J_{content}$, que mede o quanto C e G são similares e J_{style} , que deve medir o quanto o estilo das imagens G e S são similares
- Por fim, as duas funções são ponderadas por hiperparâmetros α e β
- NOTA: Pode parecer redundante usar dois hiperparâmetros para dar pesos relativos, mas essa é a convenção no artigo original
- NOTA 2: Os autores são Leon Gatys, Alexander Ecker e Matthias; Não é um artigo difícil de ler

Função de Custo

Find the generated image G

1. Initiate G randomly

$$G: \underline{100} \times \underline{100} \times \underline{3}$$

↑
RGB

2. Use gradient descent to minimize $\underline{J(G)}$

$$G := G - \frac{\partial}{\partial G} J(G)$$



Função de Custo

- A imagem G é inicializada aleatória; Em seguida se aplica o gradiente descendente para minimizar $J(G)$; Esse processo visa atualizar os valores de cada pixel na imagem G
- No exemplo da figura, com estilo de Picaso, a imagem é inicializada como um ruído branco aleatório; Em cada passo, G vai se aproximando do estilo desejado

Função de Custo

- A imagem G é inicializada aleatória; Em seguida se aplica o gradiente descendente para minimizar $J(G)$; Esse processo visa atualizar os valores de cada pixel na imagem G
- No exemplo da figura, com estilo de Picaso, a imagem é inicializada como um ruído branco aleatório; Em cada passo, G vai se aproximando do estilo desejado

Função de Custo de Conteúdo

- Lembrando que nossa função de custo é:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Falaremos agora do componente de conteúdo
- Escolhe-se uma camada da rede para calcular $J_{content}$
- Se usar uma camada muito no início, isso vai forçar a imagem gerada a usar valores muito próximos da imagem original
- Já se usar uma camada muito profunda, ele vai forçar a, se por exemplo existir um cachorro em certa posição da imagem original, também existir na imagem gerada

Função de Custo de Conteúdo

- Lembrando que nossa função de custo é:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Falaremos agora do componente de conteúdo
- Escolhe-se uma camada da rede para calcular $J_{content}$
- Se usar uma camada muito no início, isso vai forçar a imagem gerada a usar valores muito próximos da imagem original
- Já se usar uma camada muito profunda, ele vai forçar a, se por exemplo existir um cachorro em certa posição da imagem original, também existir na imagem gerada

Função de Custo de Conteúdo

- Lembrando que nossa função de custo é:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Falaremos agora do componente de conteúdo
- Escolhe-se uma camada da rede para calcular $J_{content}$
- Se usar uma camada muito no início, isso vai forçar a imagem gerada a usar valores muito próximos da imagem original
- Já se usar uma camada muito profunda, ele vai forçar a, se por exemplo existir um cachorro em certa posição da imagem original, também existir na imagem gerada

Função de Custo de Conteúdo

- Lembrando que nossa função de custo é:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Falaremos agora do componente de conteúdo
- Escolhe-se uma camada da rede para calcular $J_{content}$
- Se usar uma camada muito no início, isso vai forçar a imagem gerada a usar valores muito próximos da imagem original
- Já se usar uma camada muito profunda, ele vai forçar a, se por exemplo existir um cachorro em certa posição da imagem original, também existir na imagem gerada

Função de Custo de Conteúdo

- Lembrando que nossa função de custo é:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Falaremos agora do componente de conteúdo
- Escolhe-se uma camada da rede para calcular $J_{content}$
- Se usar uma camada muito no início, isso vai forçar a imagem gerada a usar valores muito próximos da imagem original
- Já se usar uma camada muito profunda, ele vai forçar a, se por exemplo existir um cachorro em certa posição da imagem original, também existir na imagem gerada

Função de Custo de Conteúdo

- Na prática então, escolhe-se uma camada intermediária l , nem muito rasa nem muito profunda
- Usa-se então uma rede pré-treinada (ex. VGG) e mede-se a similaridade entre G e C
- O que a parcela de conteúdo no custo mede é a similaridade (quadrado da diferença componente a componente) entre as ativações $a^{[l]}(C)$ e $a^{[l]}(G)$ na camada escolhida para G e C
- Pode haver ainda uma constante de normalização na frente, $1/2$ ou qualquer outra, não importa, já que isso vai ser ajustado pelo hiperparâmetro α

Função de Custo de Conteúdo

- Na prática então, escolhe-se uma camada intermediária l , nem muito rasa nem muito profunda
- Usa-se então uma rede pré-treinada (ex. VGG) e mede-se a similaridade entre G e C
- O que a parcela de conteúdo no custo mede é a similaridade (quadrado da diferença componente a componente) entre as ativações $a^{[l]}(C)$ e $a^{[l]}(G)$ na camada escolhida para G e C
- Pode haver ainda uma constante de normalização na frente, $1/2$ ou qualquer outra, não importa, já que isso vai ser ajustado pelo hiperparâmetro α

Função de Custo de Conteúdo

- Na prática então, escolhe-se uma camada intermediária l , nem muito rasa nem muito profunda
- Usa-se então uma rede pré-treinada (ex. VGG) e mede-se a similaridade entre G e C
- O que a parcela de conteúdo no custo mede é a similaridade (quadrado da diferença componente a componente) entre as ativações $a^{[l]}(C)$ e $a^{[l]}(G)$ na camada escolhida para G e C
- Pode haver ainda uma constante de normalização na frente, $1/2$ ou qualquer outra, não importa, já que isso vai ser ajustado pelo hiperparâmetro α

Função de Custo de Conteúdo

- Na prática então, escolhe-se uma camada intermediária l , nem muito rasa nem muito profunda
- Usa-se então uma rede pré-treinada (ex. VGG) e mede-se a similaridade entre G e C
- O que a parcela de conteúdo no custo mede é a similaridade (quadrado da diferença componente a componente) entre as ativações $a^{[l]}(C)$ e $a^{[l]}(G)$ na camada escolhida para G e C
- Pode haver ainda uma constante de normalização na frente, $1/2$ ou qualquer outra, não importa, já que isso vai ser ajustado pelo hiperparâmetro α

Função de Custo de Conteúdo

- Pensando ainda nas ativações “desenroladas” em vetores, temos o quadrado da norma L2 entre $a^{[l](C)}$ e $a^{[l](G)}$:

$$J_{content}(C, G) = \frac{1}{2} \|a^{[l](C)} - a^{[l](G)}\|^2.$$

- A minimização desse custo vai incentivar o algoritmo a obter uma imagem G que tenha as ativações na camada escondida similares às de C

Função de Custo de Conteúdo

- Pensando ainda nas ativações “desenroladas” em vetores, temos o quadrado da norma L2 entre $a^{[l](C)}$ e $a^{[l](G)}$:

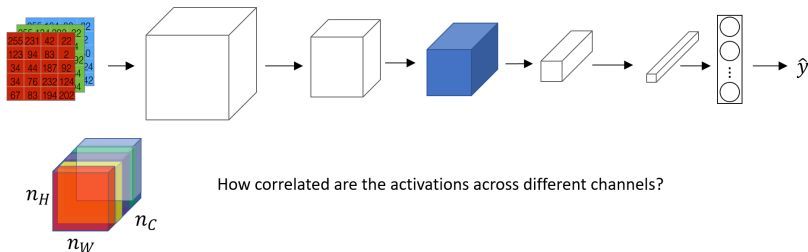
$$J_{content}(C, G) = \frac{1}{2} \|a^{[l](C)} - a^{[l](G)}\|^2.$$

- A minimização desse custo vai incentivar o algoritmo a obter uma imagem G que tenha as ativações na camada escondida similares às de C

Função de Custo de Estilo

- O que significa “estilo” em uma imagem?

Meaning of the “style” of an image

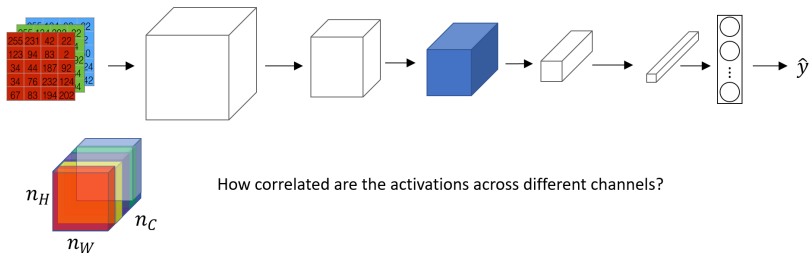


- Se olharmos para uma camada / específica, vamos definir “estilo” como sendo a correlação entre os canais da ativação da camada /
- Na figura, o bloco de ativação da camada / tem apenas 5 canais (para facilitar a visualização)

Função de Custo de Estilo

- O que significa “estilo” em uma imagem?

Meaning of the “style” of an image

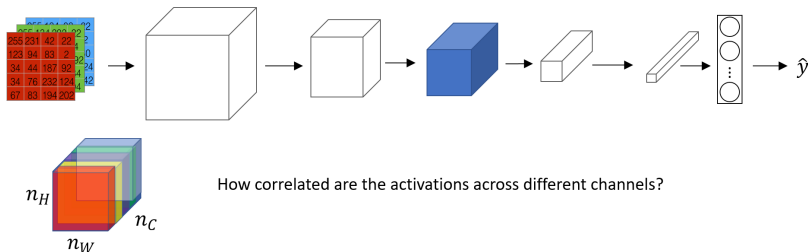


- Se olharmos para uma camada l específica, vamos definir “estilo” como sendo a correlação entre os canais da ativação da camada l
- Na figura, o bloco de ativação da camada l tem apenas 5 canais (para facilitar a visualização)

Função de Custo de Estilo

- O que significa “estilo” em uma imagem?

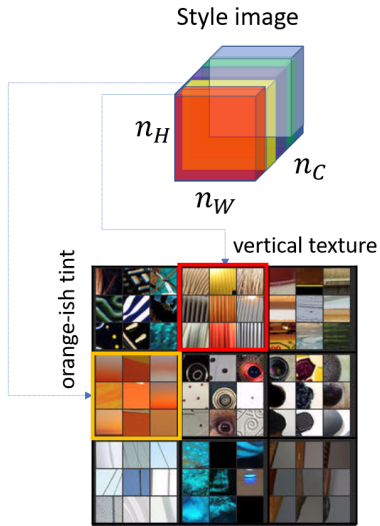
Meaning of the “style” of an image



- Se olharmos para uma camada l específica, vamos definir “estilo” como sendo a correlação entre os canais da ativação da camada l
- Na figura, o bloco de ativação da camada l tem apenas 5 canais (para facilitar a visualização)

Função de Custo de Estilo

- Foquemos nos dois primeiros canais (vermelho e amarelo)



Função de Custo de Estilo

- Em cada posição temos um par de valores (em cada canal) e calculamos a correlação desses pares
- Por que isso acaba capturando o estilo da imagem?
- Imagine na figura de exemplo que temos a correspondência entre os canais vermelho e amarelo com os neurônios cuja visualização fizemos anteriormente
- O que significa então para esses dois canais serem altamente correlacionados?
- Significa que se existe em qualquer parte da imagem uma textura vertical sutil (neurônio vermelho), essa parte vai provavelmente também conter o padrão alaranjado do neurônio amarelo

Função de Custo de Estilo

- Em cada posição temos um par de valores (em cada canal) e calculamos a correlação desses pares
- Por que isso acaba capturando o estilo da imagem?
- Imagine na figura de exemplo que temos a correspondência entre os canais vermelho e amarelo com os neurônios cuja visualização fizemos anteriormente
- O que significa então para esses dois canais serem altamente correlacionados?
- Significa que se existe em qualquer parte da imagem uma textura vertical sutil (neurônio vermelho), essa parte vai provavelmente também conter o padrão alaranjado do neurônio amarelo

Função de Custo de Estilo

- Em cada posição temos um par de valores (em cada canal) e calculamos a correlação desses pares
- Por que isso acaba capturando o estilo da imagem?
- Imagine na figura de exemplo que temos a correspondência entre os canais vermelho e amarelo com os neurônios cuja visualização fizemos anteriormente
- O que significa então para esses dois canais serem altamente correlacionados?
- Significa que se existe em qualquer parte da imagem uma textura vertical sutil (neurônio vermelho), essa parte vai provavelmente também conter o padrão alaranjado do neurônio amarelo

Função de Custo de Estilo

- Em cada posição temos um par de valores (em cada canal) e calculamos a correlação desses pares
- Por que isso acaba capturando o estilo da imagem?
- Imagine na figura de exemplo que temos a correspondência entre os canais vermelho e amarelo com os neurônios cuja visualização fizemos anteriormente
- O que significa então para esses dois canais serem altamente correlacionados?
- Significa que se existe em qualquer parte da imagem uma textura vertical sutil (neurônio vermelho), essa parte vai provavelmente também conter o padrão alaranjado do neurônio amarelo

Função de Custo de Estilo

- Em cada posição temos um par de valores (em cada canal) e calculamos a correlação desses pares
- Por que isso acaba capturando o estilo da imagem?
- Imagine na figura de exemplo que temos a correspondência entre os canais vermelho e amarelo com os neurônios cuja visualização fizemos anteriormente
- O que significa então para esses dois canais serem altamente correlacionados?
- Significa que se existe em qualquer parte da imagem uma textura vertical sutil (neurônio vermelho), essa parte vai provavelmente também conter o padrão alaranjado do neurônio amarelo

Função de Custo de Estilo

- Já se eles forem desconcorrelacionados, então se houver o padrão de texturas verticais, provavelmente não haverá o alaranjado
- Em resumo, a correlação vai dizer quais desses padrões de textura de alto nível vão ou não aparecer juntos em uma parte da imagem e medir o quanto isso acontece
- E é essa medida que vai dizer o quanto os canais da imagem gerada são correlacionados e por fim o quanto o estilo da imagem gerada é similar ao da imagem de estilo
- Dada uma imagem, vamos calcular a chamada **matriz de estilo**, que vai calcular todas essas correlações

Função de Custo de Estilo

- Já se eles forem desconcorrelacionados, então se houver o padrão de texturas verticais, provavelmente não haverá o alaranjado
- Em resumo, a correlação vai dizer quais desses padrões de textura de alto nível vão ou não aparecer juntos em uma parte da imagem e medir o quanto isso acontece
- E é essa medida que vai dizer o quanto os canais da imagem gerada são correlacionados e por fim o quanto o estilo da imagem gerada é similar ao da imagem de estilo
- Dada uma imagem, vamos calcular a chamada **matriz de estilo**, que vai calcular todas essas correlações

Função de Custo de Estilo

- Já se eles forem desconcorrelacionados, então se houver o padrão de texturas verticais, provavelmente não haverá o alaranjado
- Em resumo, a correlação vai dizer quais desses padrões de textura de alto nível vão ou não aparecer juntos em uma parte da imagem e medir o quanto isso acontece
- E é essa medida que vai dizer o quanto os canais da imagem gerada são correlacionados e por fim o quanto o estilo da imagem gerada é similar ao da imagem de estilo
- Dada uma imagem, vamos calcular a chamada **matriz de estilo**, que vai calcular todas essas correlações

Função de Custo de Estilo

- Já se eles forem desconcorrelacionados, então se houver o padrão de texturas verticais, provavelmente não haverá o alaranjado
- Em resumo, a correlação vai dizer quais desses padrões de textura de alto nível vão ou não aparecer juntos em uma parte da imagem e medir o quanto isso acontece
- E é essa medida que vai dizer o quanto os canais da imagem gerada são correlacionados e por fim o quanto o estilo da imagem gerada é similar ao da imagem de estilo
- Dada uma imagem, vamos calcular a chamada **matriz de estilo**, que vai calcular todas essas correlações

Função de Custo de Estilo - Formalização

- Definimos a notação abaixo, em que n_c é o número de canais na ativação
- Na posição $G_{kk'}^{[l]}$ temos então a correlação entre as ativações nos canais k e k'
- Define-se então:

$$a_{i,j,k}^{[l]} = \text{ativação em } (i, j, k),$$

em que i é o índice na dimensão da altura do bloco, j na largura e k no número de canais. $G^{[l]}$ é $n_c^{[l]} \times n_c^{[l]}$.

- A fórmula da matriz G é:

$$G_{kk'}^{[l]} = \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{ijk}^{[l]} a_{ijk'}^{[l]}.$$

Função de Custo de Estilo - Formalização

- Definimos a notação abaixo, em que n_c é o número de canais na ativação
- Na posição $G_{kk'}^{[l]}$ temos então a correlação entre as ativações nos canais k e k'
- Define-se então:

$$a_{i,j,k}^{[l]} = \text{ativação em } (i, j, k),$$

em que i é o índice na dimensão da altura do bloco, j na largura e k no número de canais. $G^{[l]}$ é $n_c^{[l]} \times n_c^{[l]}$.

- A fórmula da matriz G é:

$$G_{kk'}^{[l]} = \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{ijk}^{[l]} a_{ijk'}^{[l]}.$$

Função de Custo de Estilo - Formalização

- Definimos a notação abaixo, em que n_c é o número de canais na ativação
- Na posição $G_{kk'}^{[l]}$ temos então a correlação entre as ativações nos canais k e k'
- Define-se então:

$$a_{i,j,k}^{[l]} = \text{ativação em } (i, j, k),$$

em que i é o índice na dimensão da altura do bloco, j na largura e k no número de canais. $G^{[l]}$ é $n_c^{[l]} \times n_c^{[l]}$.

- A fórmula da matriz G é:

$$G_{kk'}^{[l]} = \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{ijk}^{[l]} a_{ijk'}^{[l]}.$$

Função de Custo de Estilo - Formalização

- Definimos a notação abaixo, em que n_c é o número de canais na ativação
- Na posição $G_{kk'}^{[l]}$ temos então a correlação entre as ativações nos canais k e k'
- Define-se então:

$$a_{i,j,k}^{[l]} = \text{ativação em } (i, j, k),$$

em que i é o índice na dimensão da altura do bloco, j na largura e k no número de canais. $G^{[l]}$ é $n_c^{[l]} \times n_c^{[l]}$.

- A fórmula da matriz G é:

$$G_{kk'}^{[l]} = \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{ijk}^{[l]} a_{ijk'}^{[l]}.$$

Função de Custo de Estilo - Formalização

- Note como essa correlação vai ser alta se ambas as ativações forem altas na mesma posição e baixa se ambas forem baixas
- Tecnicamente, essa medida não é uma correlação de fato, porque não subtraímos a média, mas apenas multiplicamos; Isso é chamado na verdade de covariância cruzada não-normalizada
- Essa matriz será calculada para a imagem gerada e para a de estilo
- Abaixo, usamos sobrescrito G e S para diferenciar:

$$G_{kk'}^{[l](S)} = \sum_{i=1}^{n_H^l} \sum_{j=1}^{n_W^l} a_{ijk}^{[l](S)} a_{ijk'}^{[l](S)}$$

$$G_{kk'}^{[l](G)} = \sum_{i=1}^{n_H^l} \sum_{j=1}^{n_W^l} a_{ijk}^{[l](G)} a_{ijk'}^{[l](G)}$$

Função de Custo de Estilo - Formalização

- Note como essa correlação vai ser alta se ambas as ativações forem altas na mesma posição e baixa se ambas forem baixas
- Tecnicamente, essa medida não é uma correlação de fato, porque não subtraímos a média, mas apenas multiplicamos; Isso é chamado na verdade de covariância cruzada não-normalizada
- Essa matriz será calculada para a imagem gerada e para a de estilo
- Abaixo, usamos sobrescrito G e S para diferenciar:

$$G_{kk'}^{[l](S)} = \sum_{i=1}^{n_H^l} \sum_{j=1}^{n_W^l} a_{ijk}^{[l](S)} a_{ijk'}^{[l](S)}$$

$$G_{kk'}^{[l](G)} = \sum_{i=1}^{n_H^l} \sum_{j=1}^{n_W^l} a_{ijk}^{[l](G)} a_{ijk'}^{[l](G)}$$

Função de Custo de Estilo - Formalização

- Note como essa correlação vai ser alta se ambas as ativações forem altas na mesma posição e baixa se ambas forem baixas
- Tecnicamente, essa medida não é uma correlação de fato, porque não subtraímos a média, mas apenas multiplicamos; Isso é chamado na verdade de covariância cruzada não-normalizada
- Essa matriz será calculada para a imagem gerada e para a de estilo
- Abaixo, usamos sobrescrito G e S para diferenciar:

$$G_{kk'}^{[l](S)} = \sum_{i=1}^{n_H^l} \sum_{j=1}^{n_W^l} a_{ijk}^{[l](S)} a_{ijk'}^{[l](S)}$$

$$G_{kk'}^{[l](G)} = \sum_{i=1}^{n_H^l} \sum_{j=1}^{n_W^l} a_{ijk}^{[l](G)} a_{ijk'}^{[l](G)}$$

Função de Custo de Estilo - Formalização

- Note como essa correlação vai ser alta se ambas as ativações forem altas na mesma posição e baixa se ambas forem baixas
- Tecnicamente, essa medida não é uma correlação de fato, porque não subtraímos a média, mas apenas multiplicamos; Isso é chamado na verdade de covariância cruzada não-normalizada
- Essa matriz será calculada para a imagem gerada e para a de estilo
- Abaixo, usamos sobrescrito G e S para diferenciar:

$$G_{kk'}^{[l](S)} = \sum_{i=1}^{n_H^l} \sum_{j=1}^{n_W^l} a_{ijk}^{[l](S)} a_{ijk'}^{[l](S)}$$

$$G_{kk'}^{[l](G)} = \sum_{i=1}^{n_H^l} \sum_{j=1}^{n_W^l} a_{ijk}^{[l](G)} a_{ijk'}^{[l](G)}$$

Função de Custo de Estilo - Formalização

- NOTA: Na álgebra linear, essa matriz que aqui chamamos de “estilo” é chamada na verdade de **matriz de Gram** (por isso a letra G)
- Finalmente, a função de custo de estilo é dada por:

$$\frac{1}{(2n_H^{[l]} n_W^{[l]} n_C^{[l]})^2} \|G^{[l](S)} - G^{[l](G)}\|_F^2 =$$

$$= \frac{1}{(2n_H^{[l]} n_W^{[l]} n_C^{[l]})^2} \sum_k \sum_{k'} \left(G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)} \right)^2$$

em que essa constante de normalização nem importa tanto, já que de todo modo essa parcela será multiplicada por β

- Sabe-se também que o resultado final é mais agradável quando se usa o custo de estilo de múltiplas camadas

Função de Custo de Estilo - Formalização

- NOTA: Na álgebra linear, essa matriz que aqui chamamos de “estilo” é chamada na verdade de **matriz de Gram** (por isso a letra G)
- Finalmente, a função de custo de estilo é dada por:

$$\begin{aligned} & \frac{1}{(2n_H^{[l]} n_W^{[l]} n_C^{[l]})^2} \|G^{[l](S)} - G^{[l](G)}\|_F^2 = \\ & = \frac{1}{(2n_H^{[l]} n_W^{[l]} n_C^{[l]})^2} \sum_k \sum_{k'} \left(G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)} \right)^2 \end{aligned}$$

em que essa constante de normalização nem importa tanto, já que de todo modo essa parcela será multiplicada por β

- Sabe-se também que o resultado final é mais agradável quando se usa o custo de estilo de múltiplas camadas

Função de Custo de Estilo - Formalização

- NOTA: Na álgebra linear, essa matriz que aqui chamamos de “estilo” é chamada na verdade de **matriz de Gram** (por isso a letra G)
- Finalmente, a função de custo de estilo é dada por:

$$\begin{aligned} & \frac{1}{(2n_H^{[l]} n_W^{[l]} n_C^{[l]})^2} \|G^{[l](S)} - G^{[l](G)}\|_F^2 = \\ & = \frac{1}{(2n_H^{[l]} n_W^{[l]} n_C^{[l]})^2} \sum_k \sum_{k'} \left(G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)} \right)^2 \end{aligned}$$

em que essa constante de normalização nem importa tanto, já que de todo modo essa parcela será multiplicada por β

- Sabe-se também que o resultado final é mais agradável quando se usa o custo de estilo de múltiplas camadas

Função de Custo de Estilo - Formalização

- Soma-se então o custo de estilo sobre todas as camadas, porém com hiperparâmetros $\lambda^{[l]}$, que atuam como peso para cada camada:

$$J_{style}(S, G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S, G)$$

- Isso permite que se leve em conta tanto as primeiras camadas com *features* simples de baixo nível, como bordas, até as mais profundas, com *features* de alto nível
- Juntando então as peças, voltamos à nossa função de custo original:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Esta será minimizada por gradiente descendente ou outro otimizador mais sofisticado e a imagem G que minimiza J será uma obra artística bem verossímil!

Função de Custo de Estilo - Formalização

- Soma-se então o custo de estilo sobre todas as camadas, porém com hiperparâmetros $\lambda^{[l]}$, que atuam como peso para cada camada:

$$J_{style}(S, G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S, G)$$

- Isso permite que se leve em conta tanto as primeiras camadas com *features* simples de baixo nível, como bordas, até as mais profundas, com *features* de alto nível

- Juntando então as peças, voltamos à nossa função de custo original:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Esta será minimizada por gradiente descendente ou outro otimizador mais sofisticado e a imagem G que minimiza J será uma obra artística bem verossímil!

Função de Custo de Estilo - Formalização

- Soma-se então o custo de estilo sobre todas as camadas, porém com hiperparâmetros $\lambda^{[l]}$, que atuam como peso para cada camada:

$$J_{style}(S, G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S, G)$$

- Isso permite que se leve em conta tanto as primeiras camadas com *features* simples de baixo nível, como bordas, até as mais profundas, com *features* de alto nível
- Juntando então as peças, voltamos à nossa função de custo original:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Esta será minimizada por gradiente descendente ou outro otimizador mais sofisticado e a imagem G que minimiza J será uma obra artística bem verossímil!

Função de Custo de Estilo - Formalização

- Soma-se então o custo de estilo sobre todas as camadas, porém com hiperparâmetros $\lambda^{[l]}$, que atuam como peso para cada camada:

$$J_{style}(S, G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S, G)$$

- Isso permite que se leve em conta tanto as primeiras camadas com *features* simples de baixo nível, como bordas, até as mais profundas, com *features* de alto nível
- Juntando então as peças, voltamos à nossa função de custo original:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G).$$

- Esta será minimizada por gradiente descendente ou outro otimizador mais sofisticado e a imagem G que minimiza J será uma obra artística bem verossímil!

Outline

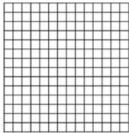
- 1 O que é Reconhecimento Facial
- 2 One-shot Learning
- 3 Redes Siamesas
- 4 Verificação Facial e Classificação Binária
- 5 Neural Style Transfer
- 6 Função de Custo
- 7 Convoluções Generalizadas para 1D e 3D**

Convolução Generalizada

- Muitas das ideias que vimos para imagens (2D) podem ser adaptadas para dados 1D e 3D
- Vimos a convolução 2D, em que podemos ter 3 canais de entrada e ainda aplicar 10 filtros
- Mas uma ideia similar se aplica em 1D

Convolução Generalizada

- Muitas das ideias que vimos para imagens (2D) podem ser adaptadas para dados 1D e 3D
- Vimos a convolução 2D, em que podemos ter 3 canais de entrada e ainda aplicar 10 filtros


 2D input image
 14×14

*


 2D filter
 5×5

$$14 \times 14 \times \underline{3} * 5 \times 5 \times \underline{3}$$

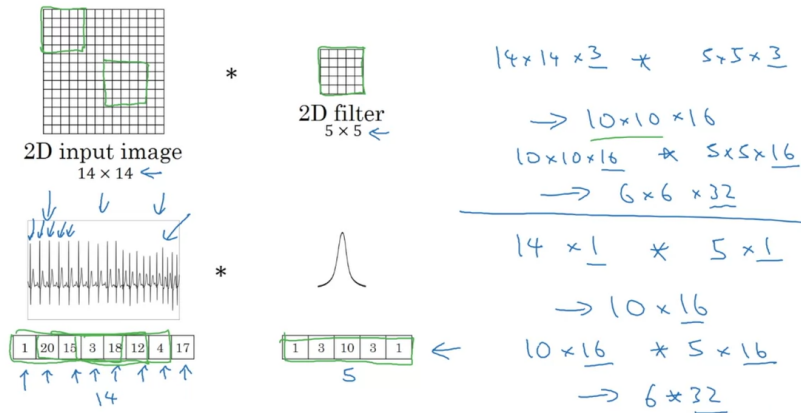
$$\rightarrow 10 \times 10 \times 16$$

- Mas uma ideia similar se aplica em 1D

Convolução Generalizada

- Muitas das ideias que vimos para imagens (2D) podem ser adaptadas para dados 1D e 3D
- Vimos a convolução 2D, em que podemos ter 3 canais de entrada e ainda aplicar 10 filtros
- Mas uma ideia similar se aplica em 1D

Convolução 1D



- No exemplo, temos à esquerda um ECG (eletrocardiograma), em que cada pico corresponde a uma batida do coração

Convolução 1D

- O ECG é uma série temporal, da voltagem no eletrodo em cada instante de tempo, e portanto é naturalmente 1D
- E esse dado pode ser convoluído com um filtro 1D
- A ideia é exatamente a mesma de ir “deslizando” o filtro sobre a entrada, multiplicando os valores sobrepostos e somando
- E novamente poderia ter múltiplos canais de entrada e múltiplos filtros na saída; Isso seria uma camada da ConvNet
- E, novamente, se a entrada da camada tiver 16 canais, o filtro também tem 16 canais

Convolução 1D

- O ECG é uma série temporal, da voltagem no eletrodo em cada instante de tempo, e portanto é naturalmente 1D
- E esse dado pode ser convoluído com um filtro 1D
- A ideia é exatamente a mesma de ir “deslizando” o filtro sobre a entrada, multiplicando os valores sobrepostos e somando
- E novamente poderia ter múltiplos canais de entrada e múltiplos filtros na saída; Isso seria uma camada da ConvNet
- E, novamente, se a entrada da camada tiver 16 canais, o filtro também tem 16 canais

Convolução 1D

- O ECG é uma série temporal, da voltagem no eletrodo em cada instante de tempo, e portanto é naturalmente 1D
- E esse dado pode ser convoluído com um filtro 1D
- A ideia é exatamente a mesma de ir “deslizando” o filtro sobre a entrada, multiplicando os valores sobrepostos e somando
- E novamente poderia ter múltiplos canais de entrada e múltiplos filtros na saída; Isso seria uma camada da ConvNet
- E, novamente, se a entrada da camada tiver 16 canais, o filtro também tem 16 canais

Convolução 1D

- O ECG é uma série temporal, da voltagem no eletrodo em cada instante de tempo, e portanto é naturalmente 1D
- E esse dado pode ser convoluído com um filtro 1D
- A ideia é exatamente a mesma de ir “deslizando” o filtro sobre a entrada, multiplicando os valores sobrepostos e somando
- E novamente poderia ter múltiplos canais de entrada e múltiplos filtros na saída; Isso seria uma camada da ConvNet
- E, novamente, se a entrada da camada tiver 16 canais, o filtro também tem 16 canais

Convolução 1D

- O ECG é uma série temporal, da voltagem no eletrodo em cada instante de tempo, e portanto é naturalmente 1D
- E esse dado pode ser convoluído com um filtro 1D
- A ideia é exatamente a mesma de ir “deslizando” o filtro sobre a entrada, multiplicando os valores sobrepostos e somando
- E novamente poderia ter múltiplos canais de entrada e múltiplos filtros na saída; Isso seria uma camada da ConvNet
- E, novamente, se a entrada da camada tiver 16 canais, o filtro também tem 16 canais

Convolução 1D

- Os filtros aqui também atuam como detectores de *features*, p.ex., detectando um padrão de ritmo cardíaco em diferentes posições
- Veremos também que muitas vezes dados 1D são processados por modelos sequenciais, mas uma ConvNet 1D também é possível e veremos mais para frente os prós e contras

Convolução 1D

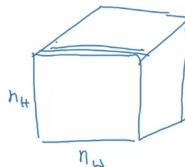
- Os filtros aqui também atuam como detectores de *features*, p.ex., detectando um padrão de ritmo cardíaco em diferentes posições
- Veremos também que muitas vezes dados 1D são processados por modelos sequenciais, mas uma ConvNet 1D também é possível e veremos mais para frente os prós e contras

Convolução 3D

- A única diferença na Conv3D é que partimos de um bloco 3D
- Veja abaixo o exemplo de uma tomografia, que escaneia o seu corpo gerando múltiplas “fatias”

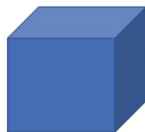
Convolução 3D

- A única diferença na Conv3D é que partimos de um bloco 3D
- Veja abaixo o exemplo de uma tomografia, que escaneia o seu corpo gerando múltiplas “fatias”

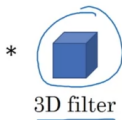


Convolução 3D

- Suponha que a entrada seja um bloco $14 \times 14 \times 14$ (só para simplificar, pois não precisa ser quadrado)



3D volume



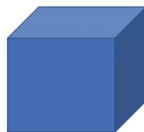
3D filter

$$\begin{array}{l}
 \downarrow \downarrow \downarrow \downarrow n_c \\
 \underline{14 \times 14 \times 14} \times \underline{1} \\
 * \underline{5 \times 5 \times 5 \times 1} \quad 16 \text{ filtros} \\
 \rightarrow 10 \times 10 \times 10 \times \underline{16} \\
 * \underline{5 \times 5 \times 5 \times 16} \\
 \rightarrow 6 \times 6 \times 6 \times 32 \quad 32 \text{ filtros}
 \end{array}$$

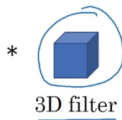
- O filtro agora também é 3D
- Tecnicamente, também podemos pensar nesse dado como tendo 1 canal ($14 \times 14 \times 14 \times 1$)

Convolução 3D

- Suponha que a entrada seja um bloco $14 \times 14 \times 14$ (só para simplificar, pois não precisa ser quadrado)



3D volume



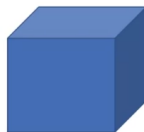
3D filter

$$\begin{array}{l}
 \downarrow \quad \downarrow \quad \downarrow \quad \downarrow^{n_c} \\
 \underline{14 \times 14 \times 14} \times \underline{1} \\
 * \underline{5 \times 5 \times 5} \times \underline{1} \quad 16 \text{ filtros} \\
 \rightarrow 10 \times 10 \times 10 \times \underline{16} \\
 * \underline{5 \times 5 \times 5} \times \underline{16} \quad 32 \text{ filtros} \\
 \rightarrow 6 \times 6 \times 6 \times 32
 \end{array}$$

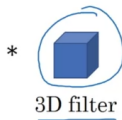
- O filtro agora também é 3D
- Tecnicamente, também podemos pensar nesse dado como tendo 1 canal ($14 \times 14 \times 14 \times 1$)

Convolução 3D

- Suponha que a entrada seja um bloco $14 \times 14 \times 14$ (só para simplificar, pois não precisa ser quadrado)



3D volume



3D filter

$$\begin{array}{l}
 \downarrow \quad \downarrow \quad \downarrow \quad \downarrow^{n_c} \\
 \underline{14 \times 14 \times 14} \times \underline{1} \\
 * \underline{5 \times 5 \times 5 \times 1} \quad 16 \text{ filter.} \\
 \rightarrow 10 \times 10 \times 10 \times \underline{16} \\
 * \underline{5 \times 5 \times 5 \times 16} \\
 \rightarrow 6 \times 6 \times 6 \times 32 \quad 32 \text{ filters}
 \end{array}$$

- O filtro agora também é 3D
- Tecnicamente, também podemos pensar nesse dado como tendo 1 canal ($14 \times 14 \times 14 \times 1$)

Convolução 3D

- E podemos ter 16 filtros; Isso seria uma camada da nossa ConvNet 3D
- E novamente temos aqui um detector de *features* no dado 3D
- Mas um outro uso muito importante da convolução 3D é sobre um filme, em que as “fatias” seriam agora *frames* no tempo
- Isso permite a detecção de ação, movimento, etc.
- As imagens são tão presentes que a imensa maioria das aplicações de ConvNets são sim em imagens, mas é importante saber dessas outras possibilidades

Convolução 3D

- E podemos ter 16 filtros; Isso seria uma camada da nossa ConvNet 3D
- E novamente temos aqui um detector de *features* no dado 3D
- Mas um outro uso muito importante da convolução 3D é sobre um filme, em que as “fatias” seriam agora *frames* no tempo
- Isso permite a detecção de ação, movimento, etc.
- As imagens são tão presentes que a imensa maioria das aplicações de ConvNets são sim em imagens, mas é importante saber dessas outras possibilidades

Convolução 3D

- E podemos ter 16 filtros; Isso seria uma camada da nossa ConvNet 3D
- E novamente temos aqui um detector de *features* no dado 3D
- Mas um outro uso muito importante da convolução 3D é sobre um filme, em que as “fatias” seriam agora *frames* no tempo
- Isso permite a detecção de ação, movimento, etc.
- As imagens são tão presentes que a imensa maioria das aplicações de ConvNets são sim em imagens, mas é importante saber dessas outras possibilidades

Convolução 3D

- E podemos ter 16 filtros; Isso seria uma camada da nossa ConvNet 3D
- E novamente temos aqui um detector de *features* no dado 3D
- Mas um outro uso muito importante da convolução 3D é sobre um filme, em que as “fatias” seriam agora *frames* no tempo
- Isso permite a detecção de ação, movimento, etc.
- As imagens são tão presentes que a imensa maioria das aplicações de ConvNets são sim em imagens, mas é importante saber dessas outras possibilidades

Convolução 3D

- E podemos ter 16 filtros; Isso seria uma camada da nossa ConvNet 3D
- E novamente temos aqui um detector de *features* no dado 3D
- Mas um outro uso muito importante da convolução 3D é sobre um filme, em que as “fatias” seriam agora *frames* no tempo
- Isso permite a detecção de ação, movimento, etc.
- As imagens são tão presentes que a imensa maioria das aplicações de ConvNets são sim em imagens, mas é importante saber dessas outras possibilidades